

MicroScopiQ: Accelerating Foundational Models through Outlier-Aware Microscaling Quantization

Akshat Ramachandran

Georgia Institute of Technology
Atlanta, USA
akshat.r@gatech.edu

Souvik Kundu

Intel Labs
San Diego, USA
souvik.kundu@intel.com

Tushar Krishna

Georgia Institute of Technology
Atlanta, USA
tushar@ece.gatech.edu

Abstract

Quantization of foundational models (FMs) is significantly more challenging than traditional DNNs due to the emergence of large magnitude values called outliers. Existing outlier-aware algorithm-architecture co-design techniques either use mixed-precision, retaining outliers at high precision but compromise hardware efficiency, or quantize inliers and outliers at the same precision, improving hardware efficiency at the cost of accuracy. To address this mutual exclusivity, we propose MicroScopiQ, a novel co-design technique that leverages pruning to complement outlier-aware quantization. MicroScopiQ retains outliers at higher precision while pruning a certain fraction of least important weights to distribute the additional outlier bits; ensuring high accuracy, aligned memory and hardware efficiency. We design a high-throughput, low overhead accelerator architecture composed of multi-precision INT processing elements and a network-on-chip called ReCoN that efficiently abstracts the complexity of supporting high-precision outliers. Additionally, unlike prior techniques, MicroScopiQ does not assume any locality of outlier weights, enabling applicability to a broad range of FMs. Extensive experiments across diverse quantization settings demonstrate that MicroScopiQ achieves state-of-the-art quantization accuracy, while delivering up to 3× faster inference and 2× lower energy consumption compared to existing alternatives. Code is available at: [MicroScopiQ-LLM-Quantization.git](https://github.com/microscopi/microscopi-llm-quantization)

CCS Concepts

• **Computer systems organization** → **Systolic arrays; Neural networks; Data flow architectures**; • **Networks** → **NoC**.

Keywords

Foundational Models, Quantization, Pruning, Hardware Accelerator

ACM Reference Format:

Akshat Ramachandran, Souvik Kundu, and Tushar Krishna. 2025. MicroScopiQ: Accelerating Foundational Models through Outlier-Aware Microscaling Quantization. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3695053.3730989>

Table 1: MicroScopiQ vs. prior outlier-aware quantization techniques, categorized into two groups, A [68, 99], B [29].

Categories	Methods		
	Group A	Group B	MicroScopiQ
Accuracy	✓ High	✗ Low	✓ High
Effective bit-width	✗ High (18.17b)	✓ Low (2b)	✓ Low (2.36b)
Flexibility	✗ No	✗ No	✓ Yes
Aligned memory	✗ Unaligned	✓ Aligned	✓ Aligned
PE design	✗ Complex	✗ Complex	✓ Simple
HW overhead	✗ High	○ Moderate	✓ Low

1 Introduction

Recent advancements in AI [9, 27, 50, 91] have been propelled by a class of models called foundational models (FMs), which encompass large language models (LLMs) and vision-language models (VLMs). FMs leverage billion-scale parameters for improved learning [96, 101] but impose substantial demands on memory, energy, and compute resources. Recent research has focused on various model compression techniques such as pruning [3, 24, 98] and quantization [53, 77, 81] to reduce memory and computational overhead, enabling efficient FM inference on resource-constrained devices.

Model pruning [24] reduces memory footprint by removing ineffectual model parameters, such as individual weights (unstructured) or blocks of weights (structured), and storing sparse tensors in a compressed format [36]. However, pruning of FMs may be infeasible due to, significant accuracy drops even at low pruning ratios [46, 97] and potential demand for compute and memory-intensive fine-tuning to regain accuracy. *Model quantization*, on the other hand, reduces the size of a target model by representing weights and/or activations at low precision [40, 45, 74, 77]. Recent works on quantization [17, 29, 76] have identified that quantizing LLMs is considerably more challenging than quantizing traditional DNNs [51, 89] due to the emergence of large magnitude features known as *outliers* [99]. These outliers significantly impact model accuracy and require specialized handling [29] compared to inliers.

To address the issue of outliers in FMs, recent algorithm/ architecture co-design techniques [29, 53, 81] have proposed different types of outlier-aware quantization. These techniques can be broadly categorized based on their outlier handling approach: **A** Maintaining outliers at higher precision compared to the inliers, or **B** Quantizing outliers at the same precision as inliers while using different data formats or scaling factors for outliers.

Techniques in group **A**, such as OWQ [48], SpQR [18], SDQ [37] (algorithm) and GOBO [99], OLAcel [68] (architecture co-design) exhibit low accuracy degradation. This is because, they typically store outliers at high precision separated from lower precision inliers. However, these techniques result in, (a) low compression



This work is licensed under a Creative Commons Attribution 4.0 International License. ISCA '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/25/06
<https://doi.org/10.1145/3695053.3730989>

factor with high *effective bit-width* (EBW¹) and (b) inefficient hardware and unaligned memory access.

On the other hand, techniques in group **B**, such as AWQ [53] (algorithm) and OliVe [29] (architecture co-design) quantize outliers at the same precision as inliers following different strategies. AWQ tries to identify a separate outlier-specific scale factor via channel-wise scaling. OliVe uses the “flint” data format [30] for inliers and “abfloat” [29] for outliers, both at 4-bit precision. These techniques mitigate the unaligned memory access while providing high compression. However, they suffer from significant accuracy degradation, particularly at ultra-low bit widths. This may be attributed to the reduced representational range available to outliers at ultra low-precision. Additionally, these methods [29] rely on a specific kind of locality of presence for outliers, that might not be true for all FMs, as we shall demonstrate in this work (§3.1).

Based on the shortcomings of existing solutions discussed above, we identify that assignment of higher bit-width for outliers is essential for good accuracy while for aligned memory and hardware efficiency a consistent *bit-budget* and data type per tensor element is desired. Here, by consistency we mean that on average each scalar within a tensor should be represented by a fixed bit-width of a particular data-type. However, these demands are conflicting.

Contributions. To provide a unified solution, we investigate on a fundamental question:

Question: *Can pruning be effectively leveraged to complement outlier-aware quantization in achieving high accuracy while maintaining hardware efficiency?*

Towards achieving this, we present a novel co-design technique for the post-training quantization (PTQ) of FMs, namely **MicroScopiQ**. Our approach effectively leverages pruning with outlier aware quantization to achieve both memory alignment and improved accuracy. To effectively perform this for a layer, we quantize outliers at twice the precision of inliers and prune the least important weights based on the Hessian information. We then redistribute the additional bits of the outlier weights in these pruned locations. This ensures memory alignment while allowing outlier weights to have higher precision. Additionally, to reduce error we use the recently proposed MicroScaling (MX) FP data format [16, 78] to quantize outlier weights as opposed to MX-INT inlier quantization. While prior work such as SDQ [37] also combined pruning and quantization, it contrasts with our approach in its limited outlier flexibility, lower compression factor, and unaligned memory access.

To efficiently support outliers in a **different** format with **different** bits at **different** locations in hardware, we present an intelligent NoC architecture called, **Redistribution and Coordination NoC** (ReCoN). It offers minimal overhead and high throughput outlier processing and reorganization. We then present an accelerator that leverages ReCoN with a simple, homogeneous INT-PE array. Additionally, we extend the accelerator to be generic enough to support multiple bit-precision (2/4-bit operations). As summarized in Table 1, MicroScopiQ blends the advantages of group **A** and **B** techniques while mitigating their specific drawbacks.

Our key contributions can be summarized as follows:

- We present MicroScopiQ, a PTQ framework to efficiently integrate pruning with outlier-aware quantization (§4).

¹The average number of bits used to represent each quantized parameter of a model.

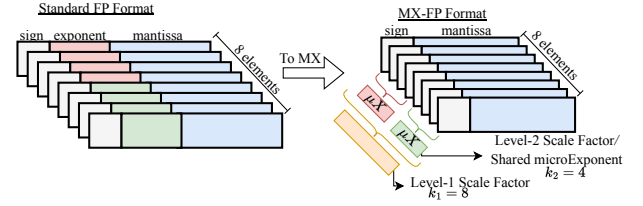


Figure 1: Depiction of MX-FP data format with level-1 scale factor and level-2 microExponent (μX), with k_1 and k_2 the group sizes over which these two factors are shared.

- To effectively deploy MicroScopiQ in a systolic array architecture we present a novel architecture supporting multi-precision, homogeneous PEs with a low-overhead NoC architecture (§5).

- To our best knowledge, MicroScopiQ is the first co-design technique, to push the limits of PTQ compression for **both** LLMs and VLMs with an EBW of **~2.36-bits** for weights; achieving SoTA quantized model accuracy across different weight/weight-activation quantization settings. Moreover, it demonstrates up to **3 $\times$ improvement in performance-per-unit-area** (TOPS/mm²) and up to **35% energy reduction** compared to existing architectures (§7).

2 Background

2.1 Model Quantization

A typical quantization [51, 74, 76] process involves two steps: establishing quantization parameters given the quantization data format (τ) and bit-width (b), and mapping the high-precision tensor to the quantized representation. For a typical symmetric quantization [103] (zero-point is 0) of a tensor X , the *scale factor* (s) is given by,

$$s = \frac{\max(X)}{\max_{\tau}^b} \quad (1)$$

$\max_{\tau}^b$ is the maximum representable value of a data format [74]. For b -bit INT quantization, $\max_{INT}^b = 2^{b-1} - 1$. After determining the quantization parameters, the quantized tensor is given by [74],

$$Q(X, s, b) = \text{clip}\left(\left\lfloor \frac{X}{s} \right\rfloor, \min_{\tau}^b, \max_{\tau}^b\right) \quad (2)$$

In model quantization, the quantization parameters can be shared at different granularity for different accuracy-overhead trade-offs. In increasing order of overheads, we have **per-tensor** quantization, wherein the scale factor is shared among all tensor elements. In **per-channel** quantization, the scale factor is shared per row/column of a tensor. Finally, in **group** quantization, the parameters are shared at a finer granularity between groups of k (64, 128 etc.) elements in a row or column. These groups are formed by dividing channels into multiple non-overlapping contiguous blocks. *In this paper, we adopt MX-INT and MX-FP quantization for inliers and outliers, respectively.*

2.2 Microscaling Data Format

The MX data format proposed by prior works [16, 78], is standardized by the Open Compute Project [70] with support from Microsoft, Intel, NVIDIA and others. As shown in Figure 1, MX is a variant of block data representation (BDR) [16] that defines a format to represent a group of values collectively using shared scale factors. It leverages multi-level, power-of-two scaling, at fine- (level-1, k_1) and ultra-fine (level-2, k_2) granularity [15, 21]. The MX data format is characterized by four components: i) scale factors (level-1, 2), ii)

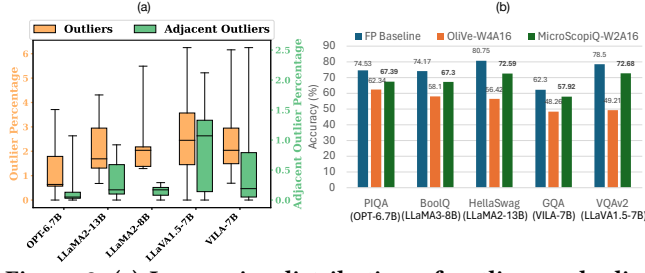


Figure 2: (a) Layer-wise distribution of outliers and adjacent outliers as a percentage of total number of weights, (b) Quantization accuracy comparison between OliVe-W4A16 and MicroScopiQ-W2A16 on various benchmarks.

data type (τ), iii) bit-width (b) and iv) group sizes (k_1, k_2). In this paper we denote an MX-FP format as $\text{MX-FP-}b_{k_1, k_2}$. In this work, we adopt the version of the MX-FP data format proposed in [78], employing multi-level scaling. The level-1 scale factor for MX-FP is computed following Equation 1. Conversely, for level-2 scale factor, we identify that MX-FP leverages the sharing of exponent field of FP values [93] (referred as μX in Figure 1). We show in §4.2 that by taking advantage of this insight i.e., the concept of shared μX , we are able to represent FP-outliers in INT format, thereby, enabling the design of simple, homogeneous INT-based PEs. For inliers, we employ $\text{MX-INT-}b_{k_1}$ with a single level of scale factor following [70]. This is because, INT format does not possess an exponent field, thereby, a level-2 scale factor similar to MX-FP is not applicable. For simplified understanding, $\text{MX-INT-}b_{k_1}$ inlier quantization can be viewed as analogous to INT group quantization utilizing an E8M0 scale factor.

3 Motivation

3.1 Limitations of existing techniques

In Table 1, we compare candidate proposals from group A: GOBO [99] and group B: OliVe [29] across various metrics. GOBO is able to achieve high accuracy by retaining outliers at full-precision. It stores outliers separately from low-precision inliers by using sparse representations with the associated outlier indices (see Figure 3(b)). By retaining outliers at full-precision, GOBO results in high EBW. Moreover, the compressed sparse storage and multiple precisions results in unaligned and random memory accesses [29], significantly impacting inference latency. Furthermore, GOBO’s outlier handling is hardware inefficient, requiring complex PEs. Similarly, a recent work [37] proposed to decompose a vector of weights into two separate inlier and outlier vectors each quantized in different precisions with outliers at a higher precision.

OliVe [29] proposes a scheme to ensure aligned memory access by quantizing inliers and outliers at the same precision (low EBW), but using different data formats. To enable differentiation between the inlier and outlier formats, it prunes the value adjacent to the outlier for use as an identifier (see Figure 3(c)). However, OliVe results in significant accuracy degradation, especially at low precision (see Figure 2(b)), due to: 1) sacrificing a number encoding from inliers for exclusive use as an identifier, reducing the number of representable values in the quantized range, and 2) the rigid assumption of outlier locality—that outliers are never adjacent to each other and only inliers are almost always adjacent to outliers (see §3.2), leading to unintended outlier pruning. Furthermore, OliVe requires

a fairly complex PE design incurring significant encoding/decoding overheads to convert the different formats into a unified processing format (exponent-integer pair). In this paper, we show that despite quantizing outliers at higher-precision and in a different format, we ensure aligned memory access, simple PE design and minimal hardware overhead.

3.2 Adjacent Outliers Matter

Similar to prior works [29, 68], we leverage the 3σ rule [71] to categorize weights as outliers. We visually demonstrate the distribution of outliers and adjacent outliers² as a percentage of the total number of weights in a layer across different FMs in Figure 2(a). As the orange box-plot shows, outliers depict a maximum percentage of $\sim 5.1\%$. Outliers are prevalent in FMs, and preserving their values is crucial for maintaining quantized model accuracy. Importantly, from the green box-plots, we observe that modern FMs on average possess $> 0.5\%$ adjacent outliers per layer, with some FM layers showing peaks of $> 2\%$. This is in stark contrast to the models evaluated by OliVe, such as BERT [19] and OPT [102] which have $< 0.04\%$ adjacent outliers (two orders of magnitude lower than FMs like LLaMA3 and LLaVA). This indicates that while pruning values adjacent to outliers could have been ideal for models like BERT [19], it is sub-optimal for most modern FMs as it removes crucial outlier values, leading to higher accuracy degradation. This is evident from Figure 2(b) where OliVe has significant accuracy degradation at 4-bit quantization due to its assumption on outlier locality. Unlike OliVe, MicroScopiQ does not naively prune adjacent values; instead it leverages the Hessian information [25] to identify the least important values to prune, ensuring outlier preservation. This directly translates to high quantized model accuracy and MicroScopiQ at 2-bit consistently outperforms OliVe across different FMs.

3.3 Outlier Precision and Data Format

The ability of group A techniques like GOBO [99] to achieve high quantized model accuracy even at extreme quantization levels of inliers (< 4 -bits) is due to retaining outliers at higher precision. This is particularly crucial at ultra-low bit width quantization because, if inliers and outliers are to be quantized to the same precision, there will be higher outlier quantization error due to the reduced representational range. We demonstrate this effect on the MicroScopiQ quantized FM accuracy in Table 7 wherein the quantized FM has poor performance when inliers and outliers are at 2-bits compared to outliers at 4-bits. Furthermore, evidence from recent work [94] demonstrates that FP-based formats for LLMs results in superior quantization performance compared to INTs. To validate this, we compare MX-INT v/s MX-FP inlier and outlier quantization in Table 7. Evidently using MX-FP instead of MX-INT for outliers results in better performance. This is due to the higher dynamic range of FPs, which is particularly beneficial at extreme quantization levels. In this work, we quantize outliers at a higher precision ($2\times$) compared to inliers, using MX-FP for outliers and MX-INT for inliers.

4 MicroScopiQ Quantization Methodology

We present an overview of MicroScopiQ quantization in Figure 3(a) and detail it in Algorithm 1. MicroScopiQ supports various group

²We define adjacent outliers as two contiguous outliers along the dot-product dimension (see row 2 of the LLM weight matrix in the center of Figure 3).

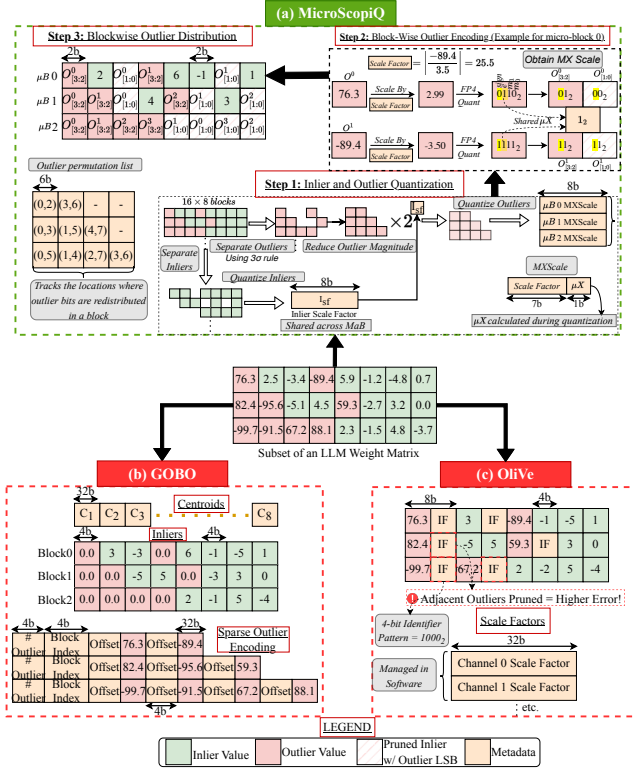


Figure 3: (a) Overview of the proposed MicroScopiQ quantization framework depicting methodology of inlier and outlier quantization and redistribution of outlier bits for a sample LLM weight matrix. Comparison against prior quantization frameworks (b) GOBO, and (c) OliVe.

size granularities and any inlier and outlier ($2\times$ inliers) data precision. For simplicity, we explain with inlier and outlier precision of 2/4- and 4/8-bit and group sizes of 128 for inliers and 8 for outliers.

4.1 Preliminaries

The MicroScopiQ quantization framework models the layer-wise post-training quantization of FMs by partitioning each layer into multiple rows and quantizing each row at a time. Concretely, for a given input calibration dataset $\mathbb{X}$, at layer l the objective is to find a quantized set of weights $\mathbb{Q} \in \mathcal{R}^{d_{row} \times d_{col}}$ that minimizes the sum of squared errors over all rows of the layer compared to the full-precision weights $\mathbb{W} \in \mathcal{R}^{d_{row} \times d_{col}}$. This can be formulated as,

$$\arg\min_{\mathbb{Q}} \sum_{i=1}^{d_{row}} \|\mathbb{W}_{i,:} \mathbb{X} - \mathbb{Q}_{i,:} \mathbb{X}\|_2^2 \quad (3)$$

We evaluate the second order derivative of Equation 3, namely the Hessian [32, 47] through Taylor series expansion [32]. Note, the Hessian is the same for all rows due to its dependence only on input data and is given as, $\mathbb{H} = 2\mathbb{X}\mathbb{X}^T$. Its inverse is, $\mathbb{H}^{-1} = (2\mathbb{X}\mathbb{X}^T + \lambda\mathbb{I})^{-1}$. We leverage the Hessian information to identify weights with small “salienices” for pruning (L17 in Algo. 1). The number of inliers to prune is determined by the number of outliers, as detailed later.

To improve quantization performance, we take inspiration from [25] and adjust the weights of unquantized rows to minimize the

Algorithm 1: MicroScopiQ Quantization Framework

```

Input :  $\mathbb{W} \in \mathcal{R}^{d_{row} \times d_{col}}$ , calibration data  $\mathbb{X}$ ,  $\mathbb{H}^{-1} = (2\mathbb{X}\mathbb{X}^T + \lambda\mathbb{I})^{-1}$ , row block
( $rB$ ), macro-block ( $MaB$ )  $B_M$ , and micro-block ( $\mu B$ )  $B_\mu$ 
Output: Quantized weight  $\mathbb{Q} \in \mathcal{R}^{d_{row} \times d_{col}}$ , perm (Permutation list)

1 # Iterate over row blocks
2 for  $i = 0, rB, 2rB, \dots, d_{col} - rB$  do
3   for  $j = i, i+1, \dots, i+rB-1$  do
4     # Step 1.0: Divide each row into non-overlapping Macro-Blocks
5     for  $\mathbb{W}_{j, MaB} \in \mathbb{W}_{j,:}$  do
6       # Step 1.1: Separate inlier and outlier in each Macro-Block
7        $\mathbb{W}_{j, MaB}^{in}, \mathbb{W}_{j, MaB}^{out} = \text{sep\_in\_out}(\mathbb{W}_{j, MaB})$ 
8       # Step 1.2: Quantize inliers to lower precision
9        $\mathbb{Q}_{j, MaB}^{in} = \text{InlierQuantization}(\mathbb{W}_{j, MaB}^{in})$ 
10      for  $\mathbb{W}_{j, \mu B} \in \mathbb{W}_{j, MaB}^{out}$  do
11        # Step 2.0: Count Number of Outliers in a Micro-Block
12         $n = \min(B_\mu/2, \text{NumOutliers}(\mathbb{W}_{j, \mu B}^{out}))$ 
13        # Step 2.1: Initialize Inlier Index List
14         $M \leftarrow \{\}$ 
15        # Step 2.2: Identify  $n$  least Important Inlier Position
16        for  $n$  iterations do
17           $p = \arg\min_{p \in \mathbb{W}_{j, MaB}^{in}} w_p^2 / [\mathbb{H}^{-1}]_{pp}$ 
18          # Step 2.3: Prune least important inliers
19           $w_p \leftarrow 0$ 
20          # Step 2.4: Update  $M$  with the location of  $w_p$ 
21           $M \leftarrow M + \{p\}$ 
22        end
23        # Step 2.5: Quantize Outliers to higher precision
24         $\mathbb{Q}_{j, \mu B}^{out}, O_{sf} = \text{OutlierQuant}(\mathbb{W}_{j, \mu B}^{out}, I_{sf})$ 
25        # Step 3.0: Distribute LSB Outlier Bits to Sparse Inlier Indices
26         $\text{perm} \leftarrow \text{DistributeOutlierBits}(\mathbb{Q}_{j, \mu B}^{out}, M)$ 
27      end
28       $\mathbb{Q}_{j, MaB} = \mathbb{Q}_{j, MaB}^{in} + \mathbb{Q}_{j, MaB}^{out}$ 
29    end
30    # Step 3.1: Quantization Error
31     $\mathbb{E}_{(j-i),:} = (\mathbb{W}_{j,:} - \mathbb{Q}_{j,:}) / [\mathbb{H}^{-1}]_{jj}$ 
32    # Step 3.2: Update weights in  $rB$  to compensate quantization error
33     $\mathbb{W}_{j:(i+rB),:} = \mathbb{W}_{j:(i+rB),:} - \mathbb{E}_{(j-i),:} \cdot \mathbb{H}_{j:(i+rB),j}^{-1}$ 
34  end
35  # Step 3.3: Update remaining weights after a row block is quantized
36   $\mathbb{W}_{(i+rB),:} = \mathbb{W}_{(i+rB),:} - \mathbb{E}_{(i+rB),:} \cdot \mathbb{H}_{(i+rB),:(i+rB)}^{-1}$ 
37 end

```

net error while quantizing a particular row of weights. The associated equation (L31 of Algo. 1) for weight update using the Hessian is derived by solving the Lagrangian of Equation 3. However, updating all remaining rows each time a row is quantized incurs significant compute-overhead, making this intractable for billion-scale FMs. Therefore, as pointed out in [25], we partition the rows into non-overlapping contiguous row-blocks (rB) of size 128 rows and localize the updates of unquantized rows within each rB . We only update the rows outside a rB (L36 in Algo. 1) once all the elements of the current block are quantized. This minimizes the number of individual updates by grouping updates together per rB and producing an order of magnitude speedup.

4.2 Inlier and Outlier Weight Quantization

In Step 1 (Figure 3(a), Algorithm 1), each row to be quantized is first divided into multiple non overlapping contiguous **macro-blocks** ($MaBs$) of size $B_M = 128$. All inliers are quantized within a MaB and share the scale factor. Each MaB is then subdivided into multiple non-overlapping contiguous **micro-blocks** (μBs) of size $B_\mu = 8$ with sixteen μBs forming a MaB . The outliers present in each μB shares same scale(s). As depicted in Figure 3(a), Step 1, the quantization process begins by first identifying inliers and outliers in each MaB by using the 3σ rule. A shared 8-bit power-of-two scale factor ($2^{I_{sf}}$), following Equation 1 is calculated for all inliers

in a MaB and the inliers are quantized to 2-bit or 4-bit, resulting in MX-INT-(2/4)₁₂₈ quantization. Interestingly, **we observe that the inlier scale factor in each MaB is always a negative power of two** for all FMs under consideration. We leverage this observation to **reduce outlier magnitude**, by multiplying all outlier values in a MaB with the inlier scale factor ($2^{I_{sf}}$) (this can also be perceived as division by $2^{-I_{sf}}$, for conformity with Equation 2). This pre-processing helps make outlier quantization easier, by pre-reducing its dynamic range before the actual outlier quantization.

Unlike inliers, outliers are quantized per μB , to reduce quantization error due to shared scaling over a larger group size (see §7). After identifying outliers present in a μB , we compute a shared 8-bit MXScale that is calculated by concatenating the level-1 power-of-two scale factor ($2^{O_{sf}^l}$) and level-2 microExponent (μX). The level-1 scale factor is calculated by following Equation 1 to obtain 7 or 5-bit MSBs of MXScale depending on size of μX being 1 or 3-bit of the LSBs—corresponds to exponent size of the FP format (depicted in Step 2 in Figure 3(a)). The outliers in a μB are scaled by ($2^{O_{sf}^l}$), following Equation 2 and then quantized to either $e1m2/e3m4$ FP-format [78] for $b = 4$ or 8-bit, respectively. Post quantization of outliers, the level-two scale factor or the μX is obtained by extracting the common exponent among all outliers in a μB . This process results in a MX-FP- $b_{8,8}$ outlier quantization. The final outlier scale factor is $2^{O_{sf}}$ where, O_{sf} is expressed as $O_{sf} = O_{sf}^l + \mu X - I_{sf}$. The term I_{sf} in the final outlier scale factor accounts for multiplication by $2^{I_{sf}}$ (or division by the inverse) during outlier pre-processing.

4.3 Outlier Value Encoding through N:M Structured Pruning

To formalize Step 2, let us assume n outliers are present in a μB quantized to MX-FP-4_{8,8}. After sharing 1-bit μX across the n outliers, each outlier has 1 sign (s) and 2 mantissa bits ($m_1 m_0$). The inliers are 2-bit 2's complement MX-INT and has 1-bit sign and 1-bit magnitude. Since the outlier bits have two mantissa bits with one sign, to ensure symmetric outlier distribution, we duplicate their sign bit and assign each sign to a mantissa creating and partitioning into two halves Upper, Lower of size 2-bit each mimicking inlier MX-INT structure i.e., $\{sm_1, sm_0\}$ (see Figure 3(a), Step 2). While we demonstrate this for a specific example, the process can be generalized to other inlier and outlier formats bit-widths wherein, the remaining outlier bits need to be reorganized into Upper and Lower distributable halves each of size bb (per element bit-budget), with bb representing the operational bit-width of each PE.

In Step 3, to ensure a fixed bb and data-type of a layer for aligned memory access and simple PE design, we distribute the outlier LSBs (Lower half) to the least important inlier locations that are pruned within a μB . For n outliers in a μB we identify n least important inlier locations to prune via the Hessian information (see Algorithm 1), forming a $(B_\mu - n):B_\mu$ structured pruning pattern³ [36, 88] i.e., $(B_\mu - n)$ non-zero values exist for every B_μ after pruning. To keep track of the corresponding halves of each of the outliers in a μB , we maintain a per- μB permutation list (Note: if there are no outliers in a μB , a permutation list is not stored). The permutation list for each μB that has outliers is made up of $\frac{B_\mu}{2}$ elements (maximum

³Following the structure of a standard N:M pattern [36] where $N = (B_\mu - n)$ and $M = B_\mu$

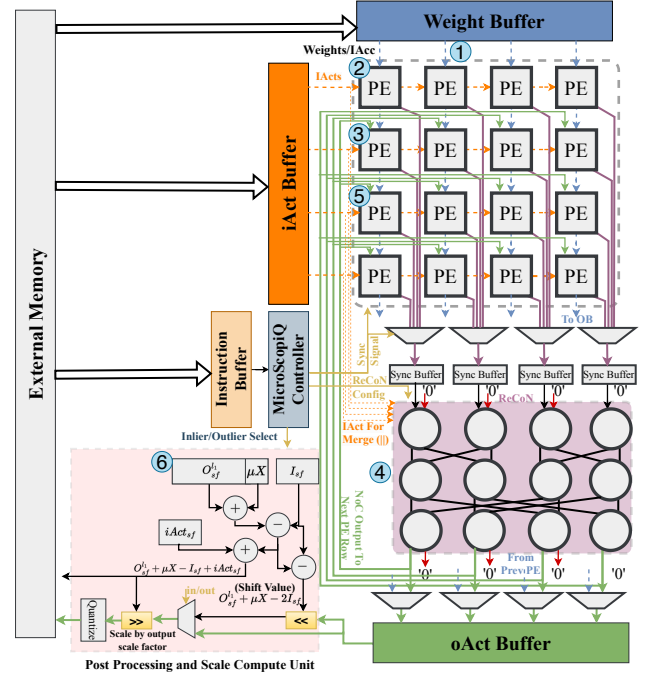


Figure 4: Integration of MicroScopiQ into a WS systolic array.

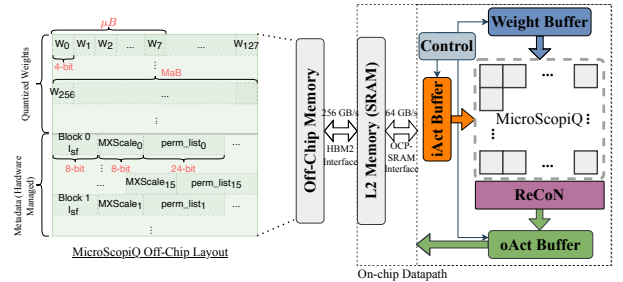


Figure 5: MicroScopiQ memory organization.

number of outliers supported per μB) with each element storing the locations of the Upper and Lower halves of outliers in a 6-bit format $\{\text{Upper}_{loc}, \text{Lower}_{loc}\}$ (see Figure 3(a) Step 3). Note that, in case of outlier count $> \frac{B_\mu}{2}$, the mB size should be chosen to be higher to prevent pruning of outliers and causing higher accuracy degradation. While such a situation does not arise in any of the models that we evaluate with $B_\mu = 8$, this feature only serves as a demonstration of the flexibility of MicroScopiQ for future models.

4.4 Effective Bit Width (EBW) Calculation

Following [25, 103] we report the EBW as the average number of bits required for storing each element including the metadata. The EBW of MicroScopiQ quantized FM varies dynamically across models and is dependent on the outlier percentage and choice of μB size. For INT-2 B_M inlier and MX-FP-4 B_μ , B_μ outlier quantization, if there are no outliers in a μB the EBW of that μB is $\text{EBW}_I = 2$, i.e., bb , whereas if there are outliers present in a mB the EBW is $\text{EBW}_O = (\text{perm}^{bits} + 2B_\mu + O_{sf}^{bits})/B_\mu$, which translates to 6-bits for $B_\mu=8$, MXScale of 8-bits and permutation list size of 24-bits (see §4.3). Since the inlier scale factor is shared across a larger group size

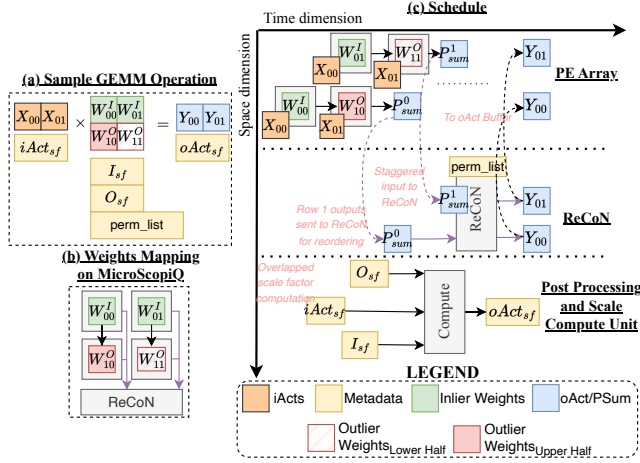


Figure 6: Example scheduling of a GEMM operation on a 2×2 MicroScopiQ accelerator.

B_M , its contribution to the EBW is negligible and hence ignored. As a rule of thumb, if there are no outlier present in a μB the EBW of that μB is equal bb else the EBW includes the contribution of all outlier-specific metadata. Additionally, the presence or absence of outlier metadata is delineated by a 1-bit identifier per μB , wherein a 1 indicates presence of outlier metadata. This identifier contributes negligible overhead to the EBW (0.05-0.09 bits) similar to the inlier scale factor and is hence ignored in the EBW calculation. For a FM with l layers and each layer having m μB s and $x\%$ of these μB s consist of outliers, the EBW of the FM is,

$$EBW_{FM} = \frac{\sum_{i=1}^l (x \cdot m \cdot EBW_O + (100 - x) \cdot m \cdot EBW_I) / m}{l} \quad (4)$$

5 MicroScopiQ Accelerator Architecture

5.1 Architecture Overview

In Figure 4, we depict the MicroScopiQ accelerator architecture, integrated into a standard weight stationary systolic array. The accelerator is optimized for throughput and all compute units (PE array, ReCoN etc.) are internally **pipelined** with interleaved pipeline stages. Figure 5 illustrates MicroScopiQ's interaction with on-chip and off-chip memories. Following prior work [15, 22, 41], off-chip memory is modeled as HBM2 with a 256 GB/s bandwidth. The off-chip memory layout [29, 99] of a MicroScopiQ-quantized tensor, shown in Figure 5, consists of two sections: quantized weights and metadata. Following prior work [29, 30, 77] a two-level on-chip memory hierarchy is employed: 2MB L2 global SRAM loads data from DRAM and transfers it to MicroScopiQ buffers (§5.2) via an Open Compute Protocol OCP-SRAM interface [86] with a bandwidth of 64 GB/s.

Through the numbered steps in Figure 4, we explain the computational flow of the MicroScopiQ accelerator. And in Figure 6(a-c), we demonstrate the mapping and schedule for a sample 2×2 GEMM operation with inlier and outlier weights, following the same computation flow.

Assume a quantized FM with outliers distributed within a μB and corresponding metadata calculated offline.

① One μB or multiple μB s of weights are mapped to each PE row as typically $B_\mu \leq \#$ of columns in PE array [39]. Each PE in a row either receives a weight at high-precision (e.g. 4-bit) or multiple packed weights at low-precision (e.g. two 2-bits).

② PE row 0 receives quantized input activation (iAct) from left and input accumulation (iAcc) from top. The PEs in row 0 perform multiplication of the stationary weights with the iActs, regardless of outlier/inlier weights. During accumulation, assuming the μB s mapped to PE row 0 do not contain outliers (see Figure 6(c)), the PEs accumulate the computed multiplication result with iAcc and direct the partial sums to PE row 1.

③ PE row 1 receives partial sums from the top and similarly performs INT multiplication between weights and iAct. As shown in the scheduling figure, in parallel, the output scale factors can be calculated (§5.5).

④ The mapped μB s to PE row 1 have outliers, the controller directs all PE outputs to ReCoN. PE row 1 offloads the accumulation to ReCoN (shown by dotted lines in Figure 6(c)). This is needed because the outlier will have its two halves distributed in different locations and mapped to different PEs. While the PEs in row 0 with inlier weights can accumulate the partial sum output, the PEs in row 1 with the Upper and Lower outlier halves cannot compute the outlier partial sum output. After receiving outputs from row 1, ReCoN reorders and calculates FP-outlier partial sum.

⑤ The controller signals PE row 2 to expect input partial sums from ReCoN and not from the previous row. If PE row 1 is the last row as in Figure 6(c), the oActs are directed to the oAct buffer.

⑥ oActs are post-processed (scaled and quantized). The post-processing of oActs does not require all the oActs to be computed. The post processing operations can be conducted as and when the oActs are generated and overlapped with the rest of the computation so as to hide computation and memory access latency overhead.

5.2 On-chip Storage and Control

Weight/Activation Organization. The input activations (iActs) are stored in the iAct buffer as 8-bit INTs. Lower-precision iActs (<8 -bit) are supported through sign-extension. The same process is followed for oActs (output activations). The weight buffer stores weights at 4-bit granularity. At lower precision (2-bits), each buffer location simultaneously stores two 2-bit weights. The **MODE** signal from the controller delineates the bit format of weights in the weight buffer i.e., one 4-bit weight or two 2-bit weights.

Instruction Buffer (IB). It stores all the metadata required for inference of MicroScopiQ quantized FMs. Particularly, the IB stores the outlier distribution permutation list i.e., configurations for ReCoN and scale factors.

MicroScopiQ Controller. The controller generates appropriate control and signals to all units. It is functionally very similar to standard systolic array controllers [84], however, with added functionality to support specific features of ReCoN (arbitration of simultaneous access by multiple rows), multi-precision PEs, post-processing unit. Furthermore, unlike traditional controllers, the MicroScopiQ controller exerts fine-grained control using handshaking signals on the streaming of iActs, iAccs into the PE array to account for the increased pipeline depth through ReCoN for rows with outliers. The controller also generates OCP-SRAM interface control signals to manage data flow between the buffers and the L2 global SRAM.

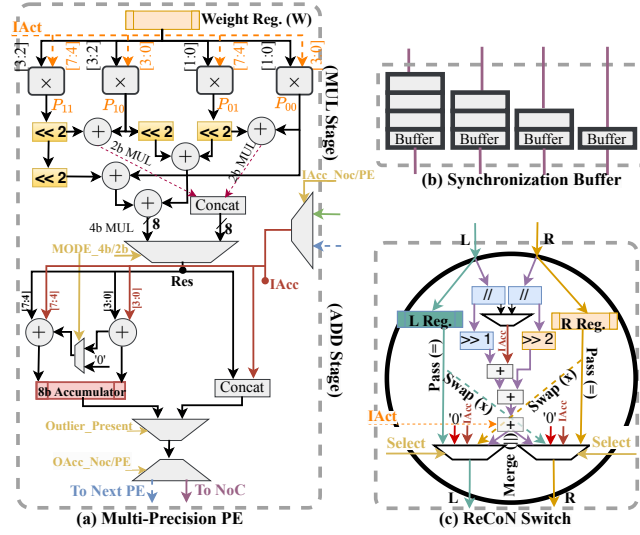


Figure 7: Microarchitecture of (a) Multi-precision PE, (b) Synchronization buffer and (c) ReCoN switch.

5.3 Multi-Precision PE Array

Multi-Precision. We depict the multi-precision MicroScopiQ PE in Figure 7(a). Existing multi-precision accelerators [29, 30, 84] typically follow a bottom-up approach, employing low-precision PEs and grouping neighboring PEs to support higher precision. This sacrifices throughput for multi-precision support, as multiple PE columns are required to perform a single MAC operation. This reduces parallelism and increases latency. We adopt a different strategy via the **MODE** signal for multi-precision support by mapping multiple weights that share the same rightbound iAct to the same PE at lower precision (2-bits) or a single weight at higher precision (4-bits). The two 2-bit weights mapped to the same PE in MicroScopiQ are the weights that would have been mapped to different columns within the same row in existing multi-precision accelerators. At high-precision operation we utilize available parallelism from the PE array while at lower-precision we increase throughput by the parallel evaluation of multiple partial sums.

Multiplication Stage. Inspired by [57, 92] we present a multiplier-tree architecture for multi-precision INT multiplication. The weights and iAct are partitioned across the four $4\text{-bit} \times 2\text{-bit}$ multipliers to calculate partial sums ($P_{00}, P_{01}, P_{10}, P_{11}$) as described in Figure 7(a). Based on the **MODE** signal the partial sums above are combined using a combination of adders and shifters to provide the result (Res) of different bit-precision weights with iAct as follows,

$$\text{Res} = \begin{cases} \{(P_{11} \ll 2 + P_{10}), (P_{01} \ll 2 + P_{00})\}, \text{MODE}_{2b} \\ (P_{11} \ll 4 + P_{00}) + (P_{01} \ll 2 + P_{10} \ll 2), \text{MODE}_{4b} \end{cases} \quad (5)$$

Accumulation Stage. It receives the MUL-stage output (Res) and also iAcc from either ReCoN or the previous PE row. The controller signals the ADD stage via **Outlier_Present** to modify accumulation behavior based on whether inlier/outlier weights are present in the PE. If the Res corresponds to a multiplication of inlier weight with iAct, then the corresponding Res is added with iAcc using two adders with a multiplexer that enables multi-precision by propagating the carry from one adder to other. In low precision **MODE**, the two adders work independently to calculate partial sums in parallel. For high precision **MODE**, both adders work together to produce

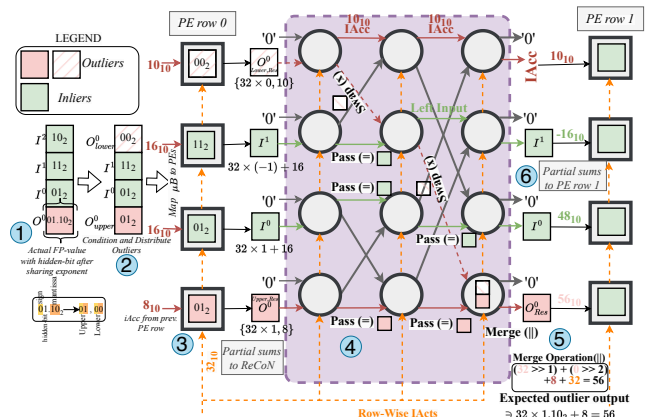


Figure 8: End-to-end example illustrating the working of MicroScopiQ with a 4×4 PE array (only two rows shown).

a single accumulation result with carry propagation through the multiplexer. On the other hand, if Res corresponds to multiplication of either of the outlier halves with iAct, the actual outlier accumulation is offloaded to ReCoN by concatenating Res and iAcc⁴. This is done to prevent incorrect outlier partial sum calculation. The controller directs partial sum outputs to ReCoN or next PE row based on presence of outliers, via the **OAcc_NoC/PE** signal.

5.4 Redistribution and Coordination NoC

The Redistribution and Coordination NoC (ReCoN) is a multistage butterfly NoC in the MicroScopiQ accelerator **time-multiplexed** and shared across PE rows (see §7.8). This is a more **cost effective** way of handling outliers compared to OliVe and GOBO [29, 99] that handle outliers within the PE. Since **PEs are large in number** OliVe, GOBO will incur larger costs compared to MicroScopiQ.

ReCoN topology. ReCoN is composed of $n(\log_2(n) + 1)$ {2-input, 2-output} ReCoN switches, in a multistage butterfly NoC topology [44]. The input and output stages of ReCoN also employ {2-input, 2-output} switches, with a dedicated switch for each column receiving (transmitting) partial sums into (from) one of the two input (output) ports. The other port of the input and output stages of each switch is tied to 0. Each ReCoN stage also receives the same iActs used by the PEs to compute the partials sums. As we shall show later, this is done to facilitate FP-outlier's hidden-bit processing.

ReCoN Input Interface. ReCoN is shared and time-multiplexed across all PE rows, and at a particular instant many rows might require access to it. At the inputs, the column-wise arbiters guided by control signals from the controller, coordinate and resolve contention between different PE rows and ensure a fair scheduling (for simplicity in Figure 4, we conceptually depict this operation as an N-input multiplexer). Due to the skewed data flow of systolic array computations [39, 77], partial sums from different columns of the same row do not arrive at the ReCoN input at the same cycle. A synchronization buffer (Figure 7(c)) is positioned between ReCoN and column-wise multiplexers which propagates the partial sums through sets of buffers of different lengths (column producing the partial sum fastest has the most buffers), synchronizing the arrival of inputs from different columns. The synchronization buffer sends an ACK signal to the PE row whose values have been accepted so

⁴the concatenated output is notated as $O_{Upper,Res}$ or $O_{Lower,Res}$.

that it can progress in its computation and the other simultaneously contesting rows will be on hold and acknowledged in the next cycle. When there are N simultaneously contesting rows, the row that is acknowledged the last adds an $N-1$ cycle latency to its processing. As we show in Figure 14(b), Figure 18(a) the num. of conflicts in accessing ReCoN are minimal and if latency is a critical factor multiple ReCoN units can be used to minimize conflicts.

ReCoN switch. It is a {2-input, 2-output} switch (Figure 7(c)) that performs three major operations based on a 3-bit configuration:

- Pass (=): Passes the input from left (right) port to left (right) output port.
- Swap ($\times$): Directs the input from the left (right) input to the right (left) output, with the opposite output port receiving the right (left) port input, 0 or iAcc (see §5.6).
- Merge ($\parallel$): This function is triggered when a switch receives $O_{Upper,Res}$ and $O_{Lower,Res}$ at its left and right ports, respectively. The switch separates the Res from iAcc (denoted as // in Figure 4) from both inputs and selects the Upper result's iAcc, since it is the correct iAcc for accumulation ($O_{Lower,Res}$ iAcc is selected during Swap). The Upper and Lower halves of outlier weight's magnitude bits are actually mantissa bits (< 1) of MX-FP which the PE treat as INTs. Therefore the Res of $O_{Upper,Res}$ and $O_{Lower,Res}$ are shifted $1\times$ and $2\times$ respectively (not shown in figure, but internally handles multi-precision **MODE**) to account for the decimal position of mantissa in the MX-FP format. The shifted Res are then accumulated with the corresponding iAcc. FP-formats also have a hidden-bit (1.0) [77, 93]; to account for the contribution of the hidden bit to the outlier partial sum we also add iAct to the accumulation above.

ReCoN Output Interface. The reordered partial sums are then routed to the subsequent row in the PE array or if the input is from the last PE row, to the oAct buffer. Since ReCoN is pipelined internally and with the rest of the PE array, reordered and processed partial sums are produced every cycle once the pipeline depth (=number of ReCoN stages) is filled.

5.5 Post Processing and Scale Compute Unit

The shared output scale factor for $B_M=128$ is calculated as $oAct_{sf} = O_{sf} + iAct_{sf}$ (See expansion of O_{sf} in §4.2.). The calculation of $oAct_{sf}$ (with simple adders and subtractors) is independent of the processing done in the PE array. Therefore, we overlap the computation of $oAct_{sf}$ with the processing of oActs to efficiently hide the computation latency similar to prior works [15, 77]. Since we maintain different scales for inliers and outliers (see §4.2), and the oAct scale factor is calculated based on the outlier scale factor (O_{sf}) (we found this to result in least quantization error of oActs), the oActs which are generated through computation with only inlier weights (identified through the in/out control signal) are shifted by the shift value depicted in the "Post Processing and Scale Compute Unit" in Figure 4 to ensure conformity with the final scale factor. Finally, the oActs are scaled with the computed output scale factor through a simple right shift (since it is a power-of-two scale factor, division can be implemented through right shift operation) and quantized to MX-INT-(4/8)₁₂₈ before being sent to external memory or routed back to the iAct buffer for computation with the next layer's weights. Similar to prior works [77], [41] the post processing unit is also responsible for handling all non-linear operations.

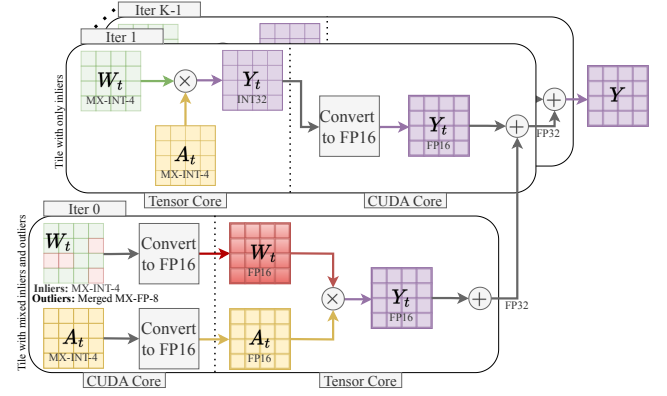


Figure 9: Illustration of W4A4 MicroScopiQ GEMM on a GPU.

5.6 MicroScopiQ Walkthrough Example

In this section, we discuss an end-to-end example that concretely showcases the working of MicroScopiQ. In the example shown in Figure 8, we employ a 4×4 systolic array (2 rows shown) with PE row 0 having one outlier and three inliers in the μB and PE row 1 having all inlier weights. Note: Not all rows require access to ReCoN, only rows that have one or more outlier weights (row 0) in the PE require access. We assume $\mu B=4$, inlier and outlier quantization formats of MX-INT-2₁₂₈ and MX-FP-4_{4,4}, respectively.

① To demonstrate that outlier partial sums processed through ReCoN results in accurate FP-outlier partial sum, we first show a μB with one outlier and its actual FP-value after sharing of μX , and having a value of 1.5_{10} . If there were no outlier distribution, the actual partial sum from an "outlier-specific PE" for the shown iAct=32₁₀ and iAcc=8₁₀ will be 56_{10} . However as described in §4.3, we distribute outliers within a μB into Upper and Lower halves. Note: The hidden-bit is implicit and is only shown for demonstration.

② The μB after distributing the outlier is mapped to PE row 0.

③ PE row 0 computes partial sum outputs for inliers in columns 1 and 2, while it outputs $O_{Upper,Res}$ and $O_{Lower,Res}$ for the other two columns that have outliers. The output of PE row 0 is routed through ReCoN. In ReCoN, the inliers are passed down at all stages, since they are already present in their respective columns.

④ The level 1 switch at column 3, executes a **swap** operation and redirects the lower half outlier output towards the upper half while simultaneously sending iAcc through the other port. This iAcc corresponds to the iAcc input to column 3 PE in row 0. This is functionally correct because, the lower half of outlier is distributed in this column by zeroing out/pruning the inlier at that location, hence the partial sum output of that PE to the corresponding PE in the next row should only be iAcc since the original weight at that location is zero. Similarly in level 2 of ReCoN, the switch that receives the $O_{Lower,Res}$ from level 1 again executes a **swap** operation. Finally in level 3 the outlier output halves are merged together to realize O_{Res}^0 , while all other switches execute pass. This merge operation results in a O_{Res}^0 that matches the expected output.

⑤, ⑥ The partial sums from ReCoN are then sent to PE row 1 which then perform MAC operations and sends it to the subsequent row since there are no outlier weights. Similarly all rows execute operations until the final partial sum is calculated.

Table 2: Quantization Results for LLMs. We report WikiText2 perplexity numbers (lower the better).

Method	W/A	OPT[102]		LLaMA-2[91]			LLaMA-3[63]		Mixtral [38]	Phi-3[2]	
		6.7B	175B	7B	13B	70B	8B	70B	8x7B	3.8B	14B
Baseline	16/16	10.86	8.34	5.47	4.83	3.31	6.13	2.85	3.84	6.33	4.31
OliVe [30]	4/16	12.20	9.09	11.52	9.34	7.23	10.29	5.65	6.19	8.57	7.81
GOBO [99]	4/16	10.97	8.71	5.79	5.03	3.45	7.11	3.53	4.22	6.64	4.78
GPTQ [25]	4/16	11.12	9.09	6.23	5.58	4.28	8.12	3.75	4.68	7.17	5.13
AWQ [53]	4/16	10.97	8.74	5.82	5.19	4.08	7.96	3.58	4.36	6.72	4.99
OmniQuant [81]	4/16	10.96	8.72	5.74	5.02	3.47	7.09	3.46	4.19	6.67	4.82
MicroScopiQ (Ours)	4/16	10.91	8.62	5.65	5.02	3.42	6.89	3.25	4.07	6.61	4.70
OliVe [29]	4/4	55.44	14.17	19.28	14.96	13.59	27.65	9.34	23.53	17.63	15.29
OmniQuant [81]	4/4	11.61	9.88	11.47	8.32	5.41	10.21	5.30	5.98	8.21	6.40
SmoothQuant [95]	4/4	19.54	17.62	20.47	15.63	17.62	29.54	19.32	37.54	18.11	15.39
Atom [103]	4/4	11.15	9.02	6.16	6.12	5.20	8.12	4.69	5.35	7.59	5.95
MicroScopiQ (Ours)	4/4	10.97	8.95	6.11	5.57	4.48	8.12	4.65	5.03	6.95	5.41
OmniQuant [81]	2/16	11.61	9.66	9.62	7.56	6.11	9.13	6.17	6.02	7.09	6.28
SDQ [37]	2/16	12.09	10.04	10.47	8.09	6.98	10.54	6.93	7.62	7.39	6.92
MicroScopiQ (Ours)	2/16	11.51	9.42	8.43	7.06	6.01	8.97	5.91	6.02	7.16	6.03
OmniQuant [81]	2/8	11.99	10.23	9.62	8.92	6.83	9.39	6.59	6.29	7.95	7.37
Atom [103]	2/8	11.95	10.13	9.23	8.54	6.33	9.13	6.35	6.14	7.46	7.29
MicroScopiQ (Ours)	2/8	11.77	9.98	9.06	8.06	6.33	9.08	6.02	6.17	7.38	6.82

6 MicroScopiQ Integration in GPUs

In this section, we demonstrate the integration of MicroScopiQ into GPUs via SW (§6.1) and HW (§6.2) modifications. For a matrix multiplication problem of size $M \times N \times K$, each GPU thread block [55, 103] is responsible for computing a $T_m \times T_n$ output tile. This computation is performed iteratively along the K dimension (Figure 9), which is referred to as the main loop. MicroScopiQ quantized models face acceleration challenges on GPUs due to: (a) high-cost pointer arithmetic from distributed outliers and (b) tensor cores' inability to co-issue INT and FP operations [14], preventing direct acceleration of tiles. Our SW, HW optimizations address these issues to maximize GPU performance.

6.1 SW: Kernel-Level Optimizations

Register Caching. We implement a virtual caching layer [7] to efficiently distribute outliers within a warp. Instead of each thread block loading and merging outliers from shared memory, we use `shfl_sync(m, r, t)` [7] for intra-warp communication. This primitive allows a thread to share its register r while reading the value from thread t within the same warp, using m as the thread selector mask. With $B_\mu = 8$, each warp (32 threads) has 4 μ Bs, allowing efficient intra-warp outlier merging based on the permutation list. **GEMM on Tensor Cores.** Due to MicroScopiQ's MX-FP format for outliers and MX-INT for inliers, mixed tiles after outlier merging (iteration 0 in Figure 9) must first be dequantized to FP16 before GEMM execution. In contrast, inlier-only tiles (iteration 1) can leverage 4-bit INT tensor cores for efficient acceleration but still require INT32-to-FP16 dequantization for accumulation along the K -dimension (Figure 9) with other tiles. We implement this block-level dynamic decision to deliver maximum performance. However, the need for repeated dequantization and lack of multi-precision support is a significant bottleneck on GPUs.

6.2 HW: Potential Tensor Core Modification

Following prior modeling [29, 73], each tensor core [1, 14] performs 16-bit FEDPs (four-element dot products). Therefore, each tensor core at 4-bits can conduct 16EDPs. Efficiently accelerating MicroScopiQ requires simultaneous INT+FP support within the tensor core. Following the functionality of ReCoN in §5.4, each 16EDP operation requires a variable right shifter (Inliers: $>> 0$, Outlier Upper Half: $>> 1$, Outlier Lower Half: $>> 2$) to account for the FP mantissa for outlier products. With typical GPU die sizes (RTX 2080 Ti: 754 mm^2), adding a shifter has negligible overhead ($\sim 0.1\%$).

7 Experimental Evaluations

7.1 Experimental Setup⁵

Models and Datasets. We evaluate on OPT [102], LLaMA2 [91] LLaMA3 [63], Phi-3 (SLM) [2] and Mixtral (MoE) [38] LLM model families. The VLMs evaluated are OpenFlamingo [5], VILA [54] and LLaVa [56]. We use a calibration dataset consisting of 256 random samples from the PILE dataset [26], so as to not overfit to any particular downstream domain [53]. We compare different LLMs based on perplexity (PPL $\downarrow$) on the WikiText2 [62] dataset and benchmark accuracy on 6 different tasks BoolQ [12], HellaSwag [100], PIQA [8], ARC-c [13], MMLU [33] and Winogrande [79].

Similarly, we evaluate VLMs on 5 vision-language benchmarks: COCO captioning [10], VQA v2 [28], VizWiz [31], TextVQA [85], and GQA [35]. Additionally, to demonstrate generalizability we also evaluate Convolutional Neural Network (CNN) baselines-ResNet50, VGG16 and State Space Model (SSM) baselines-VMamba [58], Vim [104]. For these models we employ a calibration dataset of 64 samples from the Imagenet training set [76].

⁵We denote each quantization configuration by its *bb* to enhance readability and facilitate clear comparisons.

Table 3: Comparison of quantization performance for LLaMA2-70B on LLM benchmarks.

Method	W/A	ARC-c	HellaSwag	MMLU	WinoGrande
Baseline	16/16	60.50	84.30	68.90	80.60
OliVe [29]	2/16	38.60	55.30	39.80	60.70
OmniQuant [81]	2/16	49.70	77.80	58.20	74.20
MicroScopiQ (Ours)	2/16	53.30	81.60	63.70	77.80

Algorithm Implementation. We implement the MicroScopiQ Quantization Framework in PyTorch [69]. All FMs are quantized using a single NVIDIA H100 GPU. The complete quantization process’ runtime ranges between 30 mins.–9 hours depending on model size (3.8B to 175B), which is at par with recent SoTA techniques [25, 103].

Algorithm Baselines. We compare the performance of our MicroScopiQ quantization framework against existing SoTA quantization algorithms: OmniQuant [81], AWQ [53], GPTQ [25], Atom [103], SDQ [37] and co-design techniques: OliVe [29], GOBO [99].

Accelerator Implementation. The MicroScopiQ accelerator is implemented in Verilog RTL. We perform synthesis and PnR using Synopsys Design Compiler and Innovus with a TSMC 7nm technology library (See floorplan in Figure 16(a)) for area, power and latency calculations. All MicroScopiQ design variants attains a peak clock frequency of 1 GHz. We use CACTI [65] to estimate the area and power of on-chip memories. For end-to-end performance evaluation, we design a cycle accurate simulator based on DnnWeaver [83] and BitFusion [84], following previous works [29, 77].

Accelerator Baselines. We compare the MicroScopiQ accelerator against OliVe [29], ANT [30], GOBO [99], OLAccel [68] and AdaptivFloat [89] across area, power and performance metrics. To demonstrate the minimal overhead of ReCoN on NoC-based **real-world** industrial and academic accelerators, we also implement and compare against baseline MTIA [23] and Eyeriss-v2 [11]. For a fair comparison, we ensure all designs attain a clock frequency of 1GHz and have same memory hierarchy and bandwidth. DeepScale [80] is employed to scale all designs to the 7nm process.

GPU Implementation. The optimized MicroScopiQ kernel is implemented in CUDA [103] with a PyTorch frontend. We extend GPGPU-Sim [6] and AccelSim [42] for Ampere GPUs to integrate and evaluate the modified tensor core. Energy estimation is performed using AccelWattch [43], GPUWattch [49]. Real GPU inference throughput follows the setup in [103], while simulated GPU results follow [29].

GPU Baselines. We compared system-level performance with FP16 TensorRT-LLM [66] baseline and W4A4 Atom kernel [103].

7.2 LLM Quantization Results

Weight-Only Quantization. In Table 2, we compare the WikiText2 PPL of different LLMs for W4A16 and W2A16 settings. MicroScopiQ consistently outperforms all the baselines across different models and quantization settings. At W4A16, **MicroScopiQ achieves near-lossless quantization performance**. Particularly at W2A16, the benefits of the MicroScopiQ method is evident, achieving **up to a 2.04 decrease in PPL score** compared to the baselines. This is due to MicroScopiQ’s ability to quantize outliers at higher-precision and in FP-format to reduce quantization error. At W2A16, techniques like AWQ and OliVe have unacceptable accuracy degradations ($\geq 1e4$ PPL, not depicted in table). MicroScopiQ outperforms

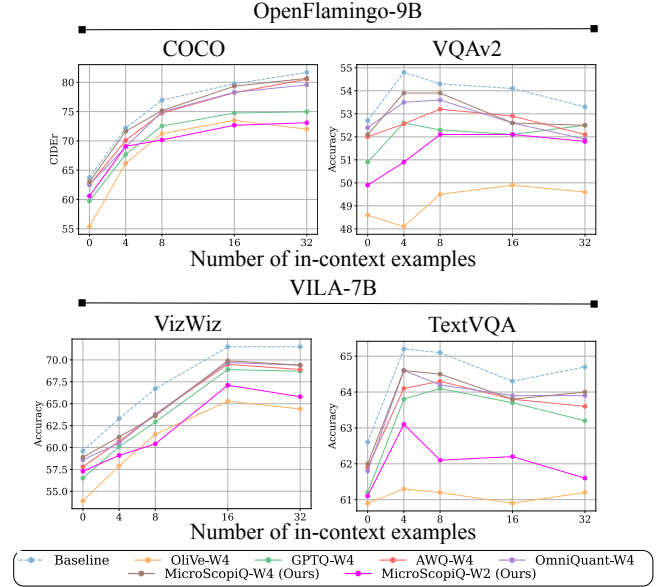


Figure 10: Weight-only quantization for VLMs, OpenFlamingo-9B [5] on COCO, VQAv2 and VILA-7B [54] on VizWiz and TextVQA.

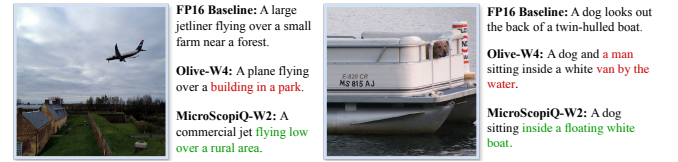


Figure 11: Qualitative results of OpenFlamingo-9B [5] on 8-shot COCO captioning with weight-only quantization using OliVe (W4) and MicroScopiQ (W2). Text in red and green indicate incorrect and accurate captioning respectively.

the contemporary SDQ [36] at W2A16 with up to a 2.06 lower PPL score, owing to its flexible pruning+quantization strategy. In contrast, SDQ relies on a rigid N:M pattern, limiting its adaptability across model families. MicroScopiQ also outperforms the SoTA quantization framework OmniQuant across all models which requires 5 – 6× **higher runtime than MicroScopiQ**.

Weight-Activation Quantization. Similarly, we compare the quantization performance of MicroScopiQ for W4A4 and W2A8 configurations in Table 2. Due to the dynamic nature of activations, very rarely do techniques quantize outliers in activations directly [103]. Instead, most techniques [81, 95] migrate the activation outliers to weights to enable simpler activation quantization. We borrow the migration strength (α) hyper-parameter introduced in SmoothQuant [95] to migrate the difficulty of quantizing outliers in activations to weights. We find that prior techniques [81, 95] can only migrate up to 50% of activation quantization difficulty to weights, i.e., $\alpha = 0.5$ before beginning to introduce higher weight quantization error. MicroScopiQ’s robustness to higher presence of outliers in weights by effectively quantizing outliers at higher precision and identifying least important weights to prune, allows α as high as 0.7 i.e., migrating most of the activation outliers to

Table 4: Quantization results for non-transformer models. We report Top-1 accuracy (%) on ImageNet (higher is better).

Method	W/A	ResNet50	VGG16	VMamba-S	Vim-S
Baseline	16/16	76.15%	71.59%	83.60%	80.50%
HAWQ [20]	2/4	73.17%	68.81%	-	-
QMamba [52]	4/4	-	-	40.36%	68.50%
MicroScopiQ	4/4	75.08%	70.84%	70.07%	71.52%
MicroScopiQ	2/8	75.12%	70.87%	66.52%	71.98%
MicroScopiQ	2/4	73.61%	69.12%	-	-

weights. This enables simpler activation quantization with MX-INT-(4/8)₁₂₈ and absolving the need to handle outliers in activations. With $\alpha = 0.7$ for MicroScopiQ, $\alpha = 0.5$ for SmoothQuant and learned migration strength for OmniQuant in Table 2, MicroScopiQ **outperforms all baselines with up to 7.4 $\times$ lower PPL score** across quantization settings. Furthermore, MicroScopiQ outperforms a recent work Atom [103] that performs activation quantization with up to a 0.33 lower PPL score.

Benchmark Accuracy. In Figure 2(b), we compare the zero-shot task accuracy of W2A16 MicroScopiQ against W416 OliVe for three different LLM benchmarks. MicroScopiQ at lower-precision than OliVe achieves **$\geq 8\%$ higher accuracy** on the benchmarks compared to OliVe. OliVe shows poor accuracy due to its assumption on outlier locality, resulting in unintended outlier pruning. In Table 3 we also compare MicroScopiQ against OliVe and OmniQuant at W2A16 setting on 4 other LLM benchmarks. MicroScopiQ consistently outperforms all baselines by up to 9% across all benchmarks.

Effective Bit Width. For $bb = 2, 4$, MicroScopiQ has an EBW of 2.36 and 4.15 bits respectively. Techniques like GOBO, have a very high EBW of 15.6, 18.17 bits. All other baselines employ software-managed metadata and have an $EBW=bb$.

7.3 VLM Quantization Results

In Figure 10, we compare 0-shot and multi-shot (4, 8, 16, 32) weight-only quantization performance of OpenFlamingo-9B on the COCO captioning task and VILA-7B on VizWiZ, TextVQA benchmarks. At W4A16 quantization, MicroScopiQ consistently outperforms all baselines and achieves on average **less than 1% accuracy drop** compared to the full-precision baseline, demonstrating the flexibility and widespread applicability of our method. Furthermore, at W2A16 quantization, MicroScopiQ achieves high accuracy ($< 4\%$ accuracy drop), outperforming several W4A16 baselines.

We also quantitatively evaluate the performance of MicroScopiQ W2A16 quantized OpenFlamingo-9B VLM on the quality of generated captions for images drawn from the COCO captioning task in Figure 11. The MicroScopiQ quantized model generates accurate captions and preserves overall word semantics compared to OliVe. For instance, in the second figure, OliVe mislabels boat as a van, whereas MicroScopiQ is able to correctly identify it as a boat.

7.4 CNN and SSM Quantization Results

In Table 4, we demonstrate MicroScopiQ quantization on CNNs and SSMs. MicroScopiQ achieves near lossless performance at W4A4, W4A8 configurations and $\leq 3\%$ accuracy drop at W2A4 setting. Similarly, MicroScopiQ achieves up to 30% higher accuracy compared to the SoTA, QMamba [52], across SSM models.

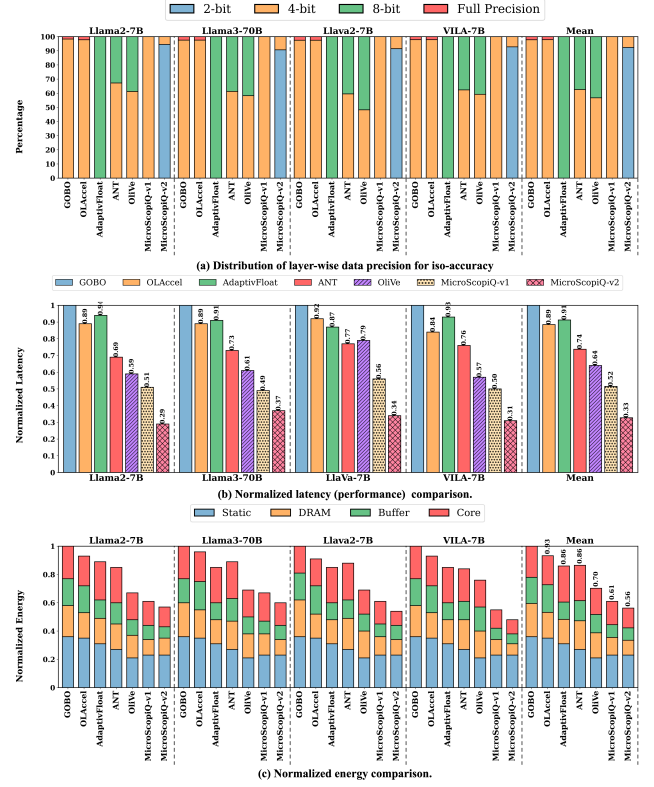


Figure 12: Iso-accuracy comparison of different accelerators. MicroScopiQ-v1 and -v2 are executed on the same accelerator and correspond to two different data-precision distributions.

7.5 Accelerator Results

Compute Area. In Table 5, we compare the accelerator compute area breakdown of MicroScopiQ (w/ 1 ReCoN unit) with baselines GOBO and OliVe for a 64×64 array design. For a fair evaluation, all accelerators have identical configurations with same number of PEs. MicroScopiQ has a very low compute overhead 8.63%, compared to OliVe's 9.90%. This is because, units like ReCoN which perform three simple operations for outlier processing has a very low area utilization compared to OliVe's encoders and decoders. ReCoN is also time-multiplexed across all rows resulting in minimal area overhead (see §7.7). Furthermore, the simple INT operations of MicroScopiQ allows the packing of more compute power with minimal overheads. A similar multi-precision design in OliVe would require 3.5-4 $\times$ higher multi-precision support area due to complex exponent-integer PEs. GOBO has higher compute area compared to OliVe and MicroScopiQ due to large per-PE area.

Maximum Performance Per Unit Area ($TOPS/mm^2$). We also estimate the peak compute throughput per unit area of each accelerator (compute density), using LLaMA3-8B as the workload in Table 5. To achieve peak throughput for MicroScopiQ, FMs must be quantized at $bb = 2$. MicroScopiQ **achieves nearly 2 $\times$ and 14 $\times$ higher compute density** than OliVe and GOBO respectively.

Iso-Accuracy Performance Comparison. To enable a fair comparison of the latency of different accelerators, we perform an iso-accuracy comparison. We quantize different weight layers of the baseline models using 2-, 4-, or 8-bit precisions, with activations

Table 5: Compute area and density comparison for a 64×64 array at 7nm process technology.

Architecture	Component (Area (μm^2), # Units)	Compute area (mm^2)	Compute overhead	Compute density (TOPS/ mm^2)
GOBO [99]	Group PE (36.56, 4096 $\times$)	0.216	3.28%	28.28
	Outlier PE (96.42, 64 $\times$)			
	Control unit (115.36, 1 $\times$)			
	4-bit Decoder (1.86, 128 $\times$)			
OliVe [29]	8-bit decoder (2.47, 64 $\times$)	0.011	9.90%	184.30
	Base PE (2.51, 4096 $\times$)			
	Multi-Precision support (0.68, 1024 $\times$)			
	Control unit (95.49, 1 $\times$)			
	ReCoN (204.68, 1 $\times$)			
MicroScopiQ	Sync buffer (20.45, 1 $\times$)	0.012	8.63%	367.51
	Base PE (2.51, 4096 $\times$)			
	Multi-precision support (0.22, 4096 $\times$)			
	Control unit (105.78, 1 $\times$)			

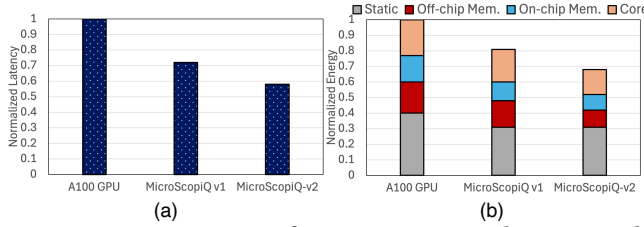


Figure 13: Comparison of MicroScopiQ accelerators with A100 GPU: (a) Normalized latency, (b) Normalized energy. at 4-bit (Figure 12(a)). This ensures all model accuracies are within $\pm 2\%$ of the best quantized model i.e., W4A4 MicroScopiQ. In Figure 12(b) we compare the normalized latency of baselines against two versions of MicroScopiQ, v1 (W4A4): all weight layers having $bb = 4$ and v2 (WxA4): most layers quantized such that $bb = 2$, with a small percentage of layers with $bb = 4$ for iso-accuracy. MicroScopiQ v1 and v2 consistently outperform all baselines across models, achieving an **average speedup of 1.50 $\times$ and 2.47 $\times$** respectively. The higher speedup of MicroScopiQ v2 is due to most layers at $bb = 2$, allowing the PEs to perform higher throughput low precision computation, compared to $bb = 4/8$.

Iso-Accuracy Energy Comparison. In Figure 12(c), we show the normalized energy consumption of different accelerators, composed of static and dynamic energy. MicroScopiQ v2 has the lowest energy consumption. Compared to designs like GOBO, OLAccel and AdaptivFloat, where computations happen at higher-precisions in 8-bit and 32-bit PEs, MicroScopiQ v2 has a significant advantage in terms of the core and DRAM dynamic energy consumption with simple INT PEs. On average MicroScopiQ v2, has **1.5 $\times$ lower energy consumption** compared to baselines across different FMs. **Power Breakdown.** The power breakdown of MicroScopiQ varies depending on the outlier distribution. For a LLaMA2-7B, the PE array consumes 56.23% of power, followed by on-chip memory (36.80%) and ReCoN (5.94%). Conversely, VILA-7B, characterized by a higher outlier percentage, exhibits increased power consumption in the ReCoN unit (7.65%) and slightly reduced on-chip memory (35.32%), PE array (55.98%) power consumption.

7.6 GPU Results

Throughput on real GPU. In Table 6, the unoptimized W4A4 MicroScopiQ (MS no-optm) underperforms the FP16 baseline due to the overhead of outlier merging in shared memory and GEMM execution in FP16. In contrast, our optimized W4A4 kernel (MS optim.) (§6.1), achieves similar performance to SoTA technique

Table 6: Normalized token generation throughput comparison across different quantization methods (Wikitext perplexity in brackets) on A100 GPU and GPU simulation [6].

Device	Method	LLaMA-2 13B	LLaMA-3 8B
A100	TRT-LLM FP16	1.00 (4.83)	1.00 (6.13)
	W4A4 Atom	2.25 (6.12)	1.05 (8.12)
	W4A4 MS* no-optm.	0.98 (5.57)	0.92 (8.12)
	W4A4 MS* optim.	2.06 (5.57)	1.01 (8.12)
GP GPU Sim	W4A4 MS* w/ New MTC [‡]	4.31 (5.57)	1.78 (8.12)

*MS: MicroScopiQ; [‡]MTC: Modified Tensor Core

Table 7: Effect on Llama3-8B PPL [63] upon progressive inclusion of different quantization techniques.

Quantization Method	WikiText2 PPL ↓
Baseline W16A16	6.13 (-)
+ Quantize all weights to INT-4	10.27 (↑ 4.14)
+ Quantize all weights to MX-INT-4 ₁₂₈	9.53 (↓ 0.74)
+ Quantize all weights to MX-INT-2 ₁₂₈	39.48 (↑ 29.95)
+ Quantize outliers to MX-FP-4 _{128,128}	10.96 (↓ 28.52)
+ Quantize outliers to MX-FP-4 _{8,8}	8.93 (↓ 2.03)
+ Reduce outlier mag. by $\times 2^{L_{sf}}$	8.89 (↓ 0.04)
+ Prune least imp. inliers per μB	9.02 (↑ 0.13)
+ Compensate quantization errors/rB	8.97 (↓ 0.05)
+ Quantize activations to MX-INT-8 ₁₂₈ , $\alpha = 0.7$	9.08 (↑ 0.11)
+ 2-bit KV-cache quantization [59]	9.58 (↑ 0.50)

Table 8: LLM quantization performance of MicroScopiQ integrated with OmniQuant.

Method	W/A	Llama-2 13B	Llama-3 70B	Phi-3 3.8B
Baseline	16/16	4.83	2.85	6.33
OmniQuant [81]	4/16	5.02	3.46	6.67
Omni-MicroScopiQ	4/16	4.87	2.97	6.52
OmniQuant [81]	2/16	7.56	6.17	7.09
Omni-MicroScopiQ	2/16	6.58	5.09	6.89
OmniQuant [81]	2/8	8.92	6.83	7.95
Omni-MicroScopiQ	2/8	7.12	5.74	7.21

Atom [103] due to efficient register caching and dynamic GEMM acceleration, while simultaneously possessing better accuracy. **MicroScopiQ GPU simulation.** As shown in our simulated A100 GPU results (Table 6), our tensor core modification enables low-precision INT+FP GEMM, eliminating costly dequantization and FP16 GEMMs, achieving the highest throughput.

GPU v/s MicroScopiQ Accelerator. We compare a **standard** A100 GPU with the MicroScopiQ accelerator under iso-bandwidth (off-chip: 2 TB/s, on-chip: 1.5kb/cycle/warp) [73] and iso-compute (55,296 multipliers [67]) scenario. As shown in Figure 13(a), MicroScopiQ v1 (W4A4) and v2 (WxA4) (see above) yield 1.2 $\times$ and 1.7 $\times$ speedup, respectively, over A100 (W4A4), due to multi-precision PEs, avoiding FP16 computation. Additionally, ReCoN’s reordering is pipelined with partial sum reduction, significantly outperforming GPU shfl_sync. Additionally, MicroScopiQ delivers superior energy efficiency (Figure 13(b)), whereas A100 incurs higher on-chip energy consumption, due to register-level reordering and FP16 overhead. *Notably, traditional GPUs lack WmAn ($m \neq n$) acceleration, making MicroScopiQ accelerator an efficient alternative.*

7.7 Algorithm ablations

Per-component accuracy impact. In Table 7 we examine the accuracy gained or lost by progressively incorporating the quantization techniques proposed in MicroScopiQ. We initially adopt

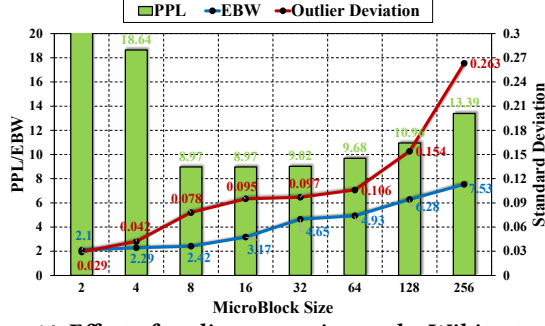


Figure 14: Effect of outlier group size on the Wikitext-v2 perplexity (PPL) and EBW of a MicroScopiQ quantized LLaMA3-8B. ∞ is per-tensor granularity.

INT-4 scalar quantization on the LLaMA3-8B full-precision baseline. We then apply MX-INT-4₁₂₈. However, on decreasing precision to 2-bit INTs i.e., MX-INT-2₁₂₈, we observe a spike in PPL, due to higher outlier quantization error. Upon quantizing outliers to MX-FP-4_{8,8}, we regain the lost performance due to preserving outlier values at higher-precision and FP-format. Furthermore, we observe that pruning of least important weights causes a minor increase in PPL, which is regained by Hessian update. We observe that quantizing activations with simple MX-INT-8₁₂₈ with a high α results in a very minimal increase in PPL due to the robustness of MicroScopiQ.

KV-cache quantization. In Table 7 we also report the impact of KV-cache quantization on performance. Following the 2-bit KV-cache quantization technique proposed in [59] we quantize K per channel, and the V per token with a MaB of size 128 and residual token length (R) [59] of 128 for both key and value.

Omni-MicroScopiQ. Our method is orthogonal to the techniques proposed in OmniQuant. We combine our method with OmniQuant to further improve the quantization performance of MicroScopiQ. We employ OmniQuant’s Learnable Weight Clipping (LWC) to learn both the inlier and outlier scale factors of MicroScopiQ and leverage the Learnable Equivalent Transformation (LET) to learn to migrate the quantization difficulty of activations to weights. In Table 8 we compare the performance of MicroScopiQ enhanced with OmniQuant (Omni-MicroScopiQ) against OmniQuant. Omni-MicroScopiQ delivers improvements up to 22%.

Outlier group size. In Figure 14, we analyze the effect of different B_μ values on outlier diversity (measure by the standard deviation: red line), PPL (green bar), and EBW (blue line) for LLaMa3-8B. For $B_\mu = 2, 4$, accuracy degradation occurs due to outlier pruning, as many mBs have $\geq B_\mu/2$ outliers. As B_μ increases, outliers further apart show greater diversity, evidenced by the higher standard deviation. For $B_\mu \geq 32$, this results in increased quantization error and higher PPL, as MX-FP scales are shared across more diverse outlier values. Larger group sizes also significantly increase EBW due to metadata overhead. A balance is achieved at $B_\mu = 8$.

7.8 Analysis of time-multiplexed ReCoN

Accelerator design variants. In Figure 15, we show the layout of different variants of MicroScopiQ with varying number of ReCoN units, each of which can be employed based on application requirements. For area critical applications, design A, with a single ReCoN can be employed to have minimal area utilization with a slight increase in latency due to access conflicts to ReCoN. With

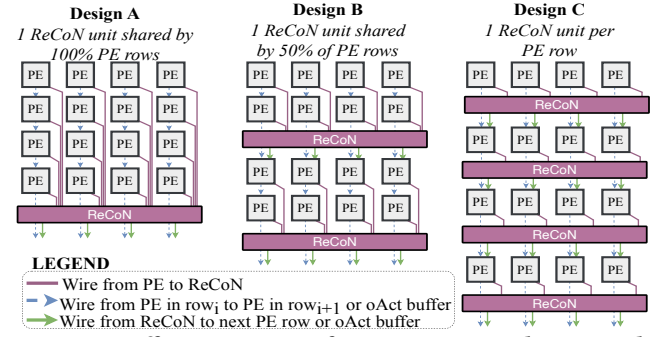


Figure 15: Different variants of MicroScopiQ accelerator with varying number of ReCoN units.

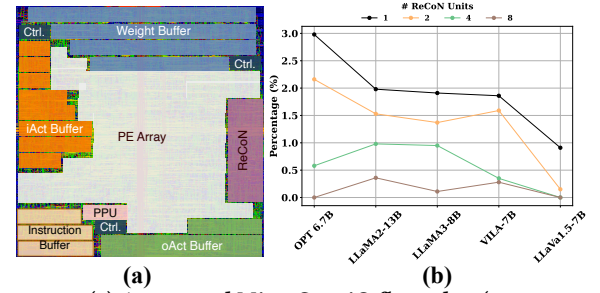


Figure 16: (a) Annotated MicroScopiQ floorplan (64×64 array). (b) Percentage access conflicts to ReCoN for 64×64 PE array.

the addition of more ReCoN units, fewer rows of PEs share a single ReCoN, dramatically reducing access conflicts to the shared ReCoN unit (design B). When the number of ReCoN units equals the number of rows, each PE row gets a dedicated ReCoN unit as shown in design C, which can be employed for latency critical applications. As we shall demonstrate subsequently, we identified that in up to a 128×128 PE array size, a total of 8 ReCoN units are sufficient to achieve peak performance with zero access conflicts to ReCoN.

Optimal # of ReCoN units. For a 64×64 MicroScopiQ accelerator, we study the number of access conflicts for different number of ReCoN units in Figure 16(b). Evidently, the number of access conflicts to ReCoN which is measured as the percent of total number of accesses to ReCoN that result in conflicts is under 3%. With the progressive addition of more ReCoN units, the access conflicts tend to 0%. As observed in Figure 12, despite the access conflicts, 1 ReCoN unit is sufficient to reach optimal inference performance in an iso-accuracy scenario. For latency critical applications, in Figure 18(a), we quantify the performance improvement and impact on compute area by incorporating multiple ReCoN units for a LLaMA3-8B FM. Up to 8 ReCoN units enables peak performance with **21% improved latency** and only **1.58 $\times$ higher compute area**. For large array sizes, up to 8 ReCoN units are sufficient for peak performance.

7.9 Scalability and Timing Analysis

Scalability. To study how MicroScopiQ scales to different PE array sizes, we compare the total area (including on-chip weight and iAct/oAct buffers) against OliVe at three different scales: 8×8 , 16×16 , 128×128 in Figure 17. Following [86], for 8×8 array we employ 16 kB iAct and oAct buffers and 32 kB weight buffer. We also depict

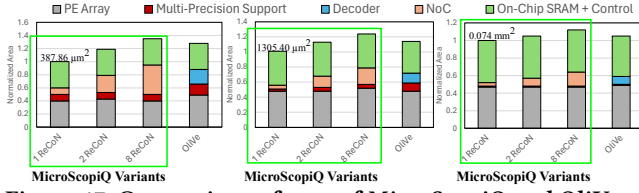


Figure 17: Comparison of area of MicroScopiQ and OliVe at different PE array sizes. Three different versions of MicroScopiQ are depicted with different number of ReCoN units.

the accelerator floorplan for a 64×64 array in Figure 16(a). We progressively scale the buffers for larger arrays to ensure full array utilization. Since varying the number of ReCoN units trades off area for performance, we also compare three versions of MicroScopiQ with 1, 2, 8 ReCoN units. For the single ReCoN variant of MicroScopiQ, as the design scales to higher array sizes, the area overhead of ReCoN decreases (128×128 has overhead of 3%), with the on-chip area being dominated by the large PE array and buffers. We observe similar trends for 2 or 8 ReCoN units. At large PE array sizes such as 128×128 , adding 8 ReCoN units results in a minimal overhead of just 11%. Particularly at large array sizes the column-wise bus from PEs to ReCoN can be routed over logic as the PE array scales. With same buffer configurations, OliVe has a higher area compared to a single ReCoN MicroScopiQ variant, while comparable area to MicroScopiQ with 8 ReCoN units. At all variants, MicroScopiQ has 2 – 3 $\times$ higher performance than OliVe (see Table 5).

Timing Analysis. MicroScopiQ at all scales attains a peak clock frequency of 1 GHz. The critical path for MicroScopiQ is from the local weights registers to multipliers within the PE. The column-wise bus (the longest wire in MicroScopiQ) from PEs to ReCoN is not on the critical path and therefore does not limit scalability.

Implementation overhead on NoC-based accelerators. The global column-wise bus is a design commonly seen in AI ASICs like [11, 23, 82, 90]. These global buses are used in conjunction with the NoC in these designs for data distribution, layout reordering [90]. We modify two large-scale accelerators MTIA and Eyeriss-v2 that employ NoCs in their design to support MicroScopiQ, by incorporating ReCoN functionality in the accelerator NoCs and modifying PEs to support all MicroScopiQ specific operations. As shown in Figure 18(b), compared to the baseline design of MTIA and Eyeriss-v2, integrating ReCoN and MicroScopiQ PEs results in only a 3% and 2.3% increase in compute area, respectively. Since these designs already employ NoCs, integration overhead of MicroScopiQ is minimal. This demonstrates that the benefits achieved by MicroScopiQ are attained at minimal to no additional cost.

8 Related Work

Model compression for LLMs. LLM weight compression schemes like quantization often rely on mixed-precision quantization with different outlier and inlier bit-widths [18, 34, 37, 48, 103]. Other class of quantization rely on channel-wise shared scaling with different scale factors for outliers [34, 48, 81]. Other quantization techniques mitigate outliers from weights and activations [4, 60, 95].

LLM weight pruning methods [3, 24, 75, 98] rely on approximate solvers to identify the importance of weights based on magnitude of weights or weights activation combined [87]. However, they fail to achieve satisfactory performance even at low sparsity levels [61].

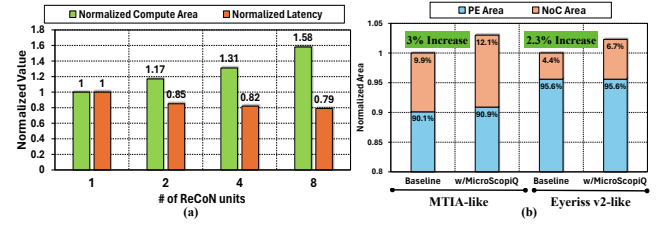


Figure 18: (a) Effect of time-multiplexed ReCoN units on the compute area and inference latency, (b) Implementation overhead of MicroScopiQ in NoC-based accelerators [11, 23].

MicroScopiQ provides a unified solution for outlier-aware quantization by enabling flexible pruning of least-important weights to distribute additional outlier bits, thereby, simultaneously achieving high accuracy and hardware efficiency.

Unified pruning and quantization. SDQ [37] combines pruning and quantization by decomposing weights into inlier and outlier vectors, each employing a fixed N:M pattern and different bit precisions. In contrast, MicroScopiQ removes the rigid N:M constraint and achieves hardware efficiency through a consistent bit-budget and unified data type per element. Moreover, unlike SDQ’s sequential pruning-then-quantization approach, MicroScopiQ integrates pruning implicitly within quantization, enabling tighter coupling.

Accelerator for quantized models. Accelerators like GOBO [99], OLAccel [68] use high-precision PEs for outliers and low-precision PEs for inliers. OliVe [29] incorporates encoding/decoding units in a PE array for inlier-outlier formats, while other accelerators [30, 77, 89] propose adaptive formats and hybrid FP PEs. However, they often suffer from unaligned memory access and large overheads. MicroScopiQ accelerator introduces a novel low overhead NoC called ReCoN, that effectively abstracts FP-format complexity from INT-PEs NoCs in ML Accelerators. NoCs are widely used in accelerators: [64, 72] employ a Benes NoC for data distribution, [90] uses a butterfly NoC for layout transformation, and [11, 23] utilize mesh NoCs. ReCoN is a butterfly NoC employed to compute FP partial sums.

9 Conclusion

In this paper, we introduce MicroScopiQ, a novel co-design technique for post-training quantization of FMs. MicroScopiQ addresses outlier-aware quantization by: a) using higher-precision MX-FP for outliers and lower-precision INTs for inliers to maintain accuracy, and b) pruning to distribute additional outlier bits, ensuring memory alignment and hardware efficiency. The MicroScopiQ accelerator, featuring multi-precision PEs and a ReCoN NoC, efficiently handles distributed outliers. MicroScopiQ is the first PTQ technique to achieve SoTA compression for both LLMs and VLMs at just 2.36-bits, offering 3 $\times$ better inference performance and 2 $\times$ lower energy consumption than existing accelerators.

Acknowledgments

This work was supported in part by CoCoSys, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA. The authors would like to thank the anonymous reviewers for their valuable feedback and suggestions, which helped improve the quality of this paper. The authors also thank Jiawei Hu for place-and-route, Jianming Tong for help with designing ReCoN, and Zishen Wan for functional accuracy simulation and constructive suggestions.

References

- [1] Hamdy Abdelkhalik, Yehia Arafa, Nandakishore Santhi, and Abdel-Hameed A Badawy. 2022. Demystifying the nvidia ampere architecture through microbenchmarking and instruction-level analysis. In *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–8.
- [2] Marah Abidin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219* (2024).
- [3] Saleh Ashkboos, Maximilian L Croci, Marcelo Gennari do Nascimento, Torsten Hoefer, and James Hensman. 2024. SliceGPT: Compress large language models by deleting rows and columns. *arXiv preprint arXiv:2401.15024* (2024).
- [4] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456* (2024).
- [5] Anas Awadalla, Irena Gao, Josh Gardner, Jack Hessel, Yusuf Hanafy, Wanrong Zhu, Kalyani Marathe, Yonatan Bitton, Samir Gadre, Shiori Sagawa, et al. 2023. Openflamingo: An open-source framework for training large autoregressive vision-language models. *arXiv preprint arXiv:2308.01390* (2023).
- [6] Ali Bakhoda, George L Yuan, Wilson WL Fung, Henry Wong, and Tor M Aamodt. 2009. Analyzing CUDA workloads using a detailed GPU simulator. In *2009 IEEE international symposium on performance analysis of systems and software*. IEEE, 163–174.
- [7] Eli Ben-Sasson, Matan Hamilis, Mark Silberstein, and Eran Tromer. 2016. Fast multiplication in binary fields on gpus via register cache. In *Proceedings of the 2016 International Conference on Supercomputing*. 1–12.
- [8] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 7432–7439.
- [9] Runjin Chen, Zhenyu Zhang, Junyuan Hong, Souvik Kundu, and Zhangyang Wang. 2024. SEAL: Steerable Reasoning Calibration of Large Language Models for Free. *arXiv preprint arXiv:2504.07986v1*.
- [10] Xinlei Chen, Hao Fang, Tsung-Yi Lin, Ramakrishna Vedantam, Saurabh Gupta, Piotr Dollár, and C Lawrence Zitnick. 2015. Microsoft coco captions: Data collection and evaluation server. *arXiv preprint arXiv:1504.00325* (2015).
- [11] Yu-Hsin Chen, Tien-Ju Yang, Joel Emer, and Vivienne Sze. 2019. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 2 (2019), 292–308.
- [12] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044* (2019).
- [13] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [14] NVIDIA Corporation. 2017. Inside Volta: The World's Most Advanced Data Center GPU. <https://devblogs.nvidia.com/inside-volta/>. Accessed: February 2025.
- [15] Steve Dai, Rangha Venkatesan, Mark Ren, Brian Zimmer, William Dally, and Bruce Khailany. 2021. Vs-quant: Per-vector scaled quantization for accurate low-precision neural network inference. *Proceedings of Machine Learning and Systems* 3 (2021), 873–884.
- [16] Bitu Darvish Rouhani, Ritchie Zhao, Venmugil Elango, Rasoul Shafipour, Mathew Hall, Maral Mesmakhoshroshahi, Ankit More, Levi Melnick, Maximilian Golub, Girish Varatkar, et al. 2023. With shared microexponents, a little shifting goes a long way. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [17] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8 () 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*. 30318–30332.
- [18] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. 2023. Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078* (2023).
- [19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [20] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. 2019. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF international conference on computer vision*. 293–302.
- [21] Mario Drumond, Tao Lin, Martin Jaggi, and Babak Falsafi. 2018. Training dnns with hybrid block floating point. *Advances in Neural Information Processing Systems* 31 (2018).
- [22] Chao Fang, Man Shi, Robin Geens, Arne Symons, Zhongfeng Wang, and Marian Verhelst. 2024. Anda: Unlocking Efficient LLM Inference with a Variable-Length Grouped Activation Data Format. *arXiv preprint arXiv:2411.15982* (2024).
- [23] Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeep Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, et al. 2023. Mta: First generation silicon targeting meta's recommendation systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [24] Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*. PMLR, 10323–10337.
- [25] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [26] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027* (2020).
- [27] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets. *Advances in neural information processing systems* 27 (2014).
- [28] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. 2017. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 6904–6913.
- [29] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2023. Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [30] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2022. Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1414–1433.
- [31] Danna Gurari, Qing Li, Abigale J Stangl, Anhong Guo, Chi Lin, Kristen Grauman, Jiebo Luo, and Jeffrey P Bigham. 2018. Vizwiz grand challenge: Answering visual questions from blind people. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 3608–3617.
- [32] Babak Hassibi, David G Stork, and Gregory J Wolff. 1993. Optimal brain surgeon and general network pruning. In *IEEE international conference on neural networks*. IEEE, 293–299.
- [33] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).
- [34] Wei Huang, Xudong Ma, Haotong Qin, Xingyu Zheng, Chengtao Lv, Hong Chen, Jie Luo, Xiaojuan Qi, Xianglong Liu, and Michele Magno. 2024. How good are low-bit quantized llama3 models? an empirical study. *arXiv preprint arXiv:2404.14047* (2024).
- [35] Drew A Hudson and Christopher D Manning. 2019. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 6700–6709.
- [36] Geonhwa Jeong, Sana Damani, Abhimanyu Rajeshkumar Bambhaniya, Eric Qin, Christopher J Hughes, Sreenivas Subramoney, Hyesoon Kim, and Tushar Krishna. 2023. Vegeta: Vertically-integrated extensions for sparse/dense gemm tile acceleration on cpus. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 259–272.
- [37] Geonhwa Jeong, Po-An Tsai, Stephen W Keckler, and Tushar Krishna. 2024. SDQ: Sparse Decomposed Quantization for LLM Inference. *arXiv preprint arXiv:2406.13868* (2024).
- [38] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [39] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [40] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. 2024. Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm. *NeurIPS ESNLP Workshop* (2024).
- [41] Ben Keller, Rangharajan Venkatesan, Steve Dai, Stephen G Tell, Brian Zimmer, Charbel Sakr, William J Dally, C Thomas Gray, and Bruce Khailany. 2023. A 95.6-TOPS/W deep learning inference accelerator with per-vector scaled 4-bit quantization in 5 nm. *IEEE Journal of Solid-State Circuits* 58, 4 (2023), 1129–1141.
- [42] Mahmoud Khairy, Jain Akshay, Tor Aamodt, and Timothy G Rogers. 2018. Exploring modern GPU memory system design challenges through accurate modeling. *arXiv preprint arXiv:1810.07269* (2018).
- [43] Mahmoud Khairy, Zhesheng Shen, Tor M Aamodt, and Timothy G Rogers. 2020. Accel-Sim: An extensible simulation framework for validated GPU modeling. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 473–486.

- [44] Tushar Krishna, Chia-Hsin Owen Chen, Woo-Cheol Kwon, and Li-Shiuan Peh. 2014. Smart: Single-cycle multi-hop traversals over a shared network on chip. *IEEE micro* 34, 3 (2014), 43–56.
- [45] Souvik Kundu, Anahita Bhiwandiwala, Sungduk Yu, Phillip Howard, Tiep Le, Sharath Nittur Sridhar, David Cobbley, Hao Kang, and Vasudev Lal. 2025. LVLm-Compress-Bench: Benchmarking the Broader Impact of Large Vision-Language Model Compression. *NACCL* (2025).
- [46] Andrey Kuzmin, Markus Nagel, Mart Van Baalen, Arash Behboodi, and Tijmen Blankevoort. 2024. Pruning vs quantization: which is better? *Advances in neural information processing systems* 36 (2024).
- [47] Yann LeCun, John Denker, and Sara Solla. 1989. Optimal brain damage. *Advances in neural information processing systems* 2 (1989).
- [48] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. 2024. Owq: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 13355–13364.
- [49] Jingwen Leng, Taylor Hetherington, Ahmed ElTantawy, Syed Gilani, Nam Sung Kim, Tor M Aamodt, and Vijay Janapa Reddi. 2013. GPUWatch: Enabling energy optimizations in GPGPUs. *ACM SIGARCH computer architecture news* 41, 3 (2013), 487–498.
- [50] Muyang Li, Ji Lin, Chenlin Meng, Stefano Ermon, Song Han, and Jun-Yan Zhu. 2022. Efficient spatially sparse inference for conditional gans and diffusion models. *Advances in neural information processing systems* 35 (2022), 28858–28873.
- [51] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. 2021. Brecq: Pushing the limit of post-training quantization by block reconstruction. *arXiv preprint arXiv:2102.05426* (2021).
- [52] Yinglong Li, Xiaoyu Liu, Jiacheng Li, Ruikang Xu, Yinda Chen, and Zhiwei Xiong. 2025. QMamba: Post-Training Quantization for Vision State Space Models. *arXiv preprint arXiv:2501.13624* (2025).
- [53] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.
- [54] Ji Lin, Hongxu Yin, Wei Ping, Pavlo Molchanov, Mohammad Shoeybi, and Song Han. 2024. Vila: On pre-training for visual language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 26689–26699.
- [55] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532* (2024).
- [56] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2024. Visual instruction tuning. *Advances in neural information processing systems* 36 (2024).
- [57] Yuhao Liu, Shubham Rai, Salim Ullah, and Akash Kumar. 2023. High Flexibility Designs of Quantized Runtime Reconfigurable Multi-Precision Multipliers. *IEEE Embedded Systems Letters* (2023).
- [58] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, Jianbin Jiao, and Yunfan Liu. 2024. VMamba: Visual State Space Model. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=ZgtLQQR1K7>
- [59] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750* (2024).
- [60] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. 2024. SpinQuant-LLM quantization with learned rotations. *arXiv preprint arXiv:2405.16406* (2024).
- [61] Xudong Lu, Aojun Zhou, Yuhui Xu, Renrui Zhang, Peng Gao, and Hongsheng Li. 2024. SPP: Sparsity-Preserved Parameter-Efficient Fine-Tuning for Large Language Models. *arXiv preprint arXiv:2405.16057* (2024).
- [62] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
- [63] AI Meta. 2024. Introducing meta llama 3: The most capable openly available llm to date. *Meta AI* (2024).
- [64] Francisco Muñoz-Martínez, Raveesh Garg, Michael Pellauer, José L Abellán, Manuel E Acacio, and Tushar Krishna. 2023. Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient dnn processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 252–265.
- [65] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to model large caches. *HP laboratories* 27 (2009), 28.
- [66] NVIDIA. 2024. TensorRT-LLM: High-Performance LLM Inference Library. <https://github.com/NVIDIA/TensorRT-LLM> Accessed: 2025-02-19.
- [67] NVIDIA. 2025. NVIDIA A100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/a100/> Accessed: 2025-02-19.
- [68] Eunhyeok Park, Dongyoung Kim, and Sungjoo Yoo. 2018. Energy-efficient neural network accelerator based on outlier-aware low-precision computation. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 688–698.
- [69] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [70] Open Compute Project. 2024. OCP Microscaling Formats MX V1.0 Spec. <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf#page=10.23> Accessed: 2024-07-13.
- [71] Friedrich Pukelsheim. 1994. The three sigma rule. *The American Statistician* 48, 2 (1994), 88–91.
- [72] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 58–70.
- [73] Md Aamir Raihan, Negar Goli, and Tor M Aamodt. 2019. Modeling deep learning accelerator enabled gpus. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 79–92.
- [74] Akshat Ramachandran, Souvik Kundu, and Tushar Krishna. 2024. CLAMP-ViT: Contrastive Data-Free Learning for Adaptive Post-Training Quantization of ViTs. *ECCV* (2024).
- [75] Akshat Ramachandran, Souvik Kundu, Arnab Raha, Shamik Kundu, Deepak K. Mathaikutty, and Tushar Krishna. 2025. Accelerating LLM Inference with Flexible N:M Sparsity via A Fully Digital Compute-in-Memory Accelerator. *arXiv:2504.14365* [cs.LG] <https://arxiv.org/abs/2504.14365>
- [76] Akshat Ramachandran, Mingyu Lee, Huan Xu, Souvik Kundu, and Tushar Krishna. 2025. OuroMamba: A Data-Free Quantization Framework for Vision Mamba Models. *arXiv preprint arXiv:2503.10959* (2025).
- [77] Akshat Ramachandran, Zishen Wan, Geonhwa Jeong, John Gustafson, and Tushar Krishna. 2024. Algorithm-Hardware Co-Design of Distribution-Aware Logarithmic-Posit Encodings for Efficient DNN Inference. *arXiv preprint arXiv:2403.05465* (2024).
- [78] Bitu Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. 2023. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537* (2023).
- [79] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106.
- [80] Satyabrata Sarangi and Bevan Baas. 2021. DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [81] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv:2308.13137* (2023).
- [82] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, et al. 2019. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 14–27.
- [83] Hardik Sharma, Jongse Park, Divya Mahajan, Emmanuel Amaro, Joon Kyung Kim, Chenkai Shao, Asit Mishra, and Hadi Esmaeilzadeh. 2016. From high-level deep neural models to FPGAs. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [84] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Joon Kyung Kim, Vikas Chandra, and Hadi Esmaeilzadeh. 2018. Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 764–775.
- [85] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. 2019. Towards vqa models that can read. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 8317–8326.
- [86] H Ekin Sumbul, Tony F Wu, Yuecheng Li, Syed Shakib Sarwar, William Koven, Eli Murphy-Trotzky, Xingxing Cai, Elnaz Ansari, Daniel H Morris, Huichu Liu, et al. 2022. System-level design and integration of a prototype AR/VR hardware featuring a custom low-power DNN accelerator chip in 7nm technology for codec avatars. In *2022 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 01–08.
- [87] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. 2023. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695* (2023).
- [88] Wei Sun, Aojun Zhou, Sander Stuijk, Rob Wijnhoven, Andrew O Nelson, Henk Corporaal, et al. 2021. DominoSearch: Find layer-wise fine-grained N: M sparse schemes from dense neural networks. *Advances in neural information processing systems* 34 (2021), 20721–20732.

- [89] Thierry Tamba, En-Yu Yang, Zishen Wan, Yuntian Deng, Vijay Janapa Reddi, Alexander Rush, David Brooks, and Gu-Yeon Wei. 2020. Algorithm-hardware co-design of adaptive floating-point encodings for resilient deep learning inference. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [90] Jianming Tong, Anirudh Itagi, Prasanth Chatarasi, and Tushar Krishna. 2024. FEATHER: A Reconfigurable Accelerator with Data Reordering Support for Low-Cost On-Chip Dataflow Switching. *arXiv preprint arXiv:2405.13170* (2024).
- [91] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [92] Yaman Umuroglu, Lahiru Rasnayake, and Magnus Sjölander. 2018. Bismo: A scalable bit-serial matrix multiplication overlay for reconfigurable computing. In *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 307–3077.
- [93] Naigang Wang, Jungwook Choi, Daniel Brand, Chia-Yu Chen, and Kailash Gopalakrishnan. 2018. Training deep neural networks with 8-bit floating point numbers. *Advances in neural information processing systems* 31 (2018).
- [94] Xiaoxia Wu, Haojun Xia, Stephen Youn, Zhen Zheng, Shiyang Chen, Arash Bakhtiari, Michael Wyatt, Yuxiong He, Olatunji Ruwase, Leon Song, et al. 2023. Zeroquant (4+ 2): Redefining llms quantization with a new fp6-centric strategy for diverse generative tasks. *arXiv preprint arXiv:2312.08583* (2023).
- [95] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
- [96] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Shaochen Zhong, Bing Yin, and Xia Hu. 2024. Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data* 18, 6 (2024), 1–32.
- [97] Lu Yin, Ajay K. Jaiswal, Shiwei Liu, Souvik Kundu, and Zhangyang Wang. 2024. Junk DNA Hypothesis: Pruning Small Pre-Trained Weights Irreversibly and Monotonically Impairs “Difficult” Downstream Tasks in LLMs. *Forty-first International Conference on Machine Learning*.
- [98] Lu Yin, You Wu, Zhenyu Zhang, Cheng-Yu Hsieh, Yaqing Wang, Yiling Jia, Mykola Pechenizkiy, Yi Liang, Zhangyang Wang, and Shiwei Liu. 2023. Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity. *arXiv preprint arXiv:2310.05175* (2023).
- [99] Ali Hadi Zadeh, Isak Edo, Omar Mohamed Awad, and Andreas Moshovos. 2020. Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 811–824.
- [100] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).
- [101] Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. 2024. When scaling meets llm finetuning: The effect of data, model and finetuning method. *arXiv preprint arXiv:2402.17193* (2024).
- [102] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
- [103] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-bit quantization for efficient and accurate llm serving. *Proceedings of Machine Learning and Systems* 6 (2024), 196–209.
- [104] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. 2024. Vision mamba: Efficient visual representation learning with bidirectional state space model. *arXiv preprint arXiv:2401.09417* (2024).